

华宸AI报告单：AIGC 原创性检测报告单

报告单编号：1779177087284

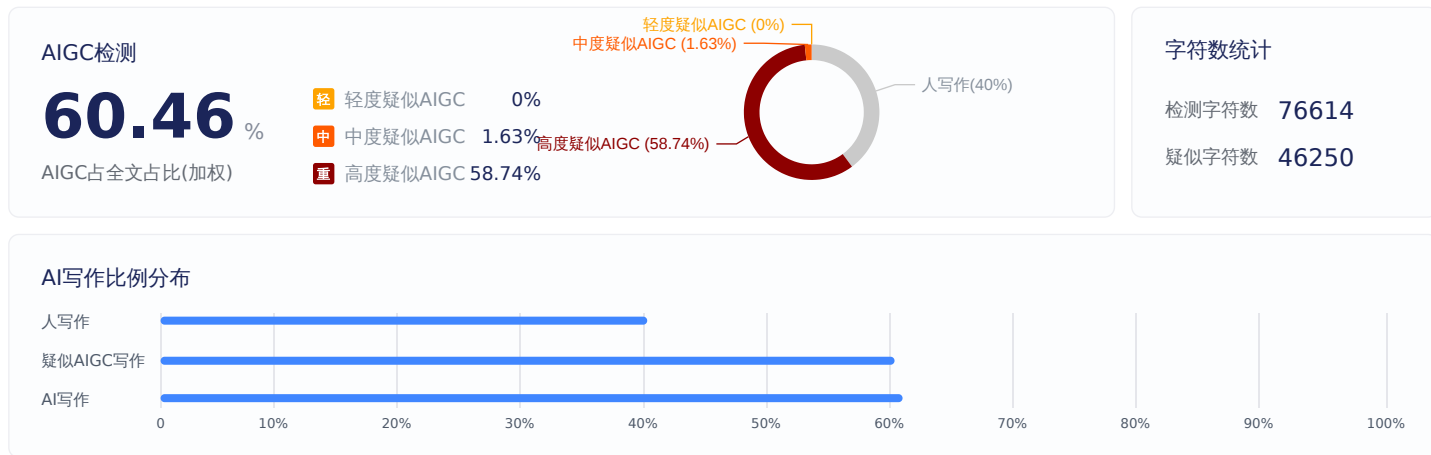
上传时间：2026-05-19

采用先进的深度学习和自然语言处理技术，以大语言模型为核心，结合特征向量分析、模型匹配算法和语义相似度测算，多层次语义分析和对抗性样本筛查等技术手段，精准识别区分人类撰写与模型生成内容，确保信息真实性

论文篇名 基于Unity3D的卡牌自走棋游戏设计与实现

作者 唐鑫阳

AIGC检测结果



论文原文

论文题目 基于Unity3D的卡牌自走棋游戏设计与实现 学院名称 软件学院 专业名称 数字媒体技术 学生姓名 唐鑫阳 指导教师 高天寒 李青山 二〇二六年六月二〇二六年六月 学号 20226980 密级 东北大学本科毕业论文 基于Unity3D的卡牌自走棋游戏设计与实现 学院名称：软件学院 专业名称：数字媒体技术 学生姓名：唐鑫阳 指导教师：高天寒 教授 李青山 高级工程师 二〇二六年六月 基于Unity3D的卡牌自走棋游戏设计与实现 作者姓名：唐鑫阳 校内指导教师：高天寒 教授 校外指导教师：李青山 高级工程师 单位名称：软件学院 专业名称：数字媒体技术 东北大学 2026年6月 Design and Implementation of a Card-Based Auto Chess Game Based on Unity3D by Tang Xinyang Supervisor: Professor Gao Tianhan Associate Supervisor: Senior Engineer Li Qingshan Northeastern University June 2026 郑重声明 本人呈交的学位论文，是在导师的指导下，独立进行研究工作所取得的成果，所有数据、图片资料真实可靠。尽我所知，除文中已经注明引用的内容外，本学位论文的研究成果不包含他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确的方式标明。本学位论文的知识产权归属于培养单位。 本人签名： 日期：2026年4月4日 摘要

随着游戏产业的快速发展，融合多种玩法机制的复合型游戏逐渐成为市场主流。然而，现有的多玩法融合游戏在系统架构设计、玩法协同以及系统可扩展性方面仍存在不足。本文针对这些问题，设计并实现了一款基于Unity引擎的多玩法融合游戏——Clash of Gods，该游戏融合了角色扮演游戏（RPG）、肉鸽（Roguelite）、搜打撤（Extraction）、自走棋（Auto Battler）和卡牌策略五种玩法机制。本文采用GameFramework框架作为底层架构，构建了一套完整的游戏系统。在系统设计上，采用了事件驱动架构和模块化设计思想，实现了战斗系统、物品背包系统、策略卡系统、探索系统等核心模块。其中，战斗系统创新性地引入了自走棋机制，实现了棋子AI自动战斗、多种命中检测方式和Buff效果系统；物品背包系统支持多种物品类型、堆叠机制和拖拽交互；策略卡系统通过配置表驱动的方式实现了灵活的卡牌效果；探索系统实现了敌人AI状态机和战斗触发机制。在技术实现方面，本文采用数据驱动的设计模式，通过Excel配置表管理游戏数据，实现了策划与程序的分离。系统采用UniTask进行异步编程，使用DOTween实现UI动画效果，通过对象池技术优化性能。经过系统测试，游戏在功能完整性、性能表现和用户体验方面均达到预期目标。本研究为多玩法融合游戏的开发提供了一套可行的技术方案，对游戏系统架构设计、模块化开发和性能优化具有一定的参考价值。

关键词：多玩法融合游戏；Unity引擎；GameFramework框架；自走棋；肉鸽；系统架构设计 ABSTRACT

With the rapid development of the game industry, hybrid games combining multiple gameplay mechanics have gradually become mainstream. However, existing hybrid games still have deficiencies in system architecture design, gameplay synergy, and system scalability. This paper addresses these issues by designing and implementing a multi-genre hybrid game called Clash of Gods based on the Unity engine, which integrates five gameplay mechanics: RPG, Roguelite, Extraction, Auto Battler, and Card Strategy.

This paper adopts the GameFramework as the underlying architecture to build a complete game system. In terms of system design, an event-driven architecture and modular design approach are adopted, implementing core modules including the combat system, inventory system, strategy card system, and

高度疑似
AIGC
(99.88%)

高度疑似
AIGC
(99.23%)

中度疑似
AIGC
(75.45%)

exploration system. Among them, the combat system innovatively introduces an auto-battler mechanism, achieving automatic chess piece AI combat, multiple hit detection methods, and a Buff effect system. The inventory system supports multiple item types, stacking mechanisms, and drag-and-drop interactions. The strategy card system implements flexible card effects through configuration table-driven approaches. The exploration system implements enemy AI state machines and combat trigger mechanisms.

In terms of technical implementation, this paper adopts a data-driven design pattern, managing game data through Excel configuration tables, achieving separation between game designers and programmers. The system uses UniTask for asynchronous programming, DOTween for UI animation effects, and object pool technology for performance optimization. After system testing, the game achieves expected goals in functional completeness, performance, and user experience. This research provides a feasible technical solution for the development of multi-genre hybrid games and has certain reference value for game system architecture design, modular development, and performance optimization.

Key words: Multi-genre Hybrid Game; Unity Engine; GameFramework; Auto Battler; Roguelite; System Architecture Design 目录 目录摘要 XI ABSTRACT XII 1 绪论 1.1 研究背景及意义 1.1.1 多元融合游戏类型发展现状与挑战 1.1.2 游戏开发技术发展趋势与研究意义 1.1.3 研究意义与贡献 2 1.1.4 现有游戏的不足与本研究的目标 3 1.2 国内外研究现状 3 1.2.1 游戏市场现状与融合型游戏的成功案例 3 1.2.2 融合型游戏的设计特点与启示 4 1.2.3 游戏开发技术与框架现状 4 1.2.4 学术研究现状与本研究的定位 5 1.3 主要研究内容 5 1.3.1 研究内容 5 1.3.2 研究方法 6 1.3.3 技术路线 6 1.4 本文的组织结构 7 2 相关技术 9 2.1 Unity游戏引擎 9 2.1.1 游戏引擎概述 9 2.1.2 Unity引擎特点 9 2.1.3 Unity核心概念 9 2.2 GameFramework框架 10 2.2.1 框架概述 10 2.2.2 核心模块 10 2.2.3 框架优势 11 2.3 多玩法融合游戏设计 11 2.3.1 主要玩法类型介绍 11 2.3.2 多玩法融合设计要素 12 2.3.3 多玩法融合游戏优势 12 2.4 系统架构设计模式 12 2.4.1 事件驱动架构 12 2.4.2 MVC模式 13 2.4.3 模块化设计 13 2.4.4 组件化设计 13 2.5 配置表驱动设计 14 2.5.1 配置表概念 14 2.5.2 配置表优势 14 2.5.3 配置表设计原则 14 2.6 本章小结 14 3 需求分析 17 3.1 业务需求 17 3.2 用户需求 17 3.3 功能需求分析 18 3.3.1 游戏概述 18 3.3.2 探索功能需求 20 3.3.3 战斗功能需求 20 3.3.4 物品管理需求 20 3.3.5 卡牌策略需求 21 3.4 非功能需求分析 21 3.5 用例分析 21 3.5.1 参与者识别 21 3.5.2 用例描述 21 3.6 可行性分析 26 3.6.1 技术可行性 26 3.6.2 经济可行性 26 3.6.3 时间可行性 26 3.7 本章小结 26 4 系统设计 29 4.1 系统总体架构设计 29 4.1.1 分层架构设计 29 4.1.2 系统整体架构图 30 4.1.3 核心系统模块划分 31 4.1.4 事件系统设计 31 4.2 战斗系统设计 32 4.2.1 战斗系统整体架构 32 4.2.2 棋子实体系统设计 34 4.2.3 Buff系统设计 36 4.2.4 技能系统设计 37 4.2.5 AI决策系统设计 40 4.2.6 伤害计算系统设计 42 4.2.7 战斗触发与脱战机制设计 43 4.3 物品与背包系统设计 44 4.3.1 物品系统整体架构 44 4.3.2 物品基础类设计 45 4.3.3 背包容器系统设计 45 4.3.4 物品堆叠机制设计 47 4.3.5 拖拽交互系统设计 47 4.3.6 词条与羁绊系统设计 47 4.4 策略卡系统设计 48 4.4.1 卡牌系统整体架构 48 4.4.2 卡牌数据管理设计 49 4.4.3 卡牌效果执行引擎设计 49 4.4.4 卡牌池管理设计 52 4.5 探索系统设计 52 4.5.1 探索系统整体架构 52 4.5.2 敌人AI状态机设计 53 4.5.3 视野检测系统设计 54 4.5.4 净化与奖励系统设计 55 4.6 数据表与配置系统设计 55 4.6.1 配置表工作流设计 55 4.6.2 核心配置表设计 56 4.6.3 配置表访问接口设计 57 4.7 UI系统设计 57 4.7.1 UI系统整体架构 57 4.7.2 UI组件引用管理设计 58 4.7.3 背包UI设计 58 4.7.4 战斗UI设计 58 4.7.5 战前准备UI设计 59 4.7.6 战斗结算UI设计 59 4.7.7 主菜单UI设计 60 4.7.8 UI动画设计 60 4.8 本章小结 60 5 系统实现 61 5.1 技术美术系统实现 61 5.1.1 渲染管线配置 61 5.1.2 着色器系统实现 61 5.1.3 粒子特效系统实现 63 5.1.4 动画系统实现 63 5.2 战斗系统实现 64 5.2.1 战斗流程管理实现 64 5.2.2 棋子管理器实现 64 5.2.3 Buff管理器实现 65 5.2.4 技能工厂与技能实现 65 5.2.5 AI决策算法实现 66 5.2.6 伤害计算公式实现 66 5.2.7 脱战系统实现 67 5.3 物品系统实现 67 5.3.1 物品工厂与物品类实现 67 5.3.2 背包管理器实现 68 5.3.3 装备系统实现 69 5.3.4 拖拽交互系统实现 69 5.3.5 物品效果执行实现 70 5.4 卡牌系统实现 71 5.4.1 卡牌管理器实现 71 5.4.2 卡牌效果执行器实现 71 5.4.3 卡牌UI实现 71 5.5 探索系统实现 72 5.5.1 敌人AI状态机实现 72 5.5.2 视野检测系统实现 72 5.5.3 战斗触发器实现 72 5.6 数据配置系统实现 73 5.6.1 配置表加载实现 73 5.6.2 配置缓存机制实现 73 5.6.3 配置表通用访问接口实现 74 5.7 性能优化策略 75 5.7.1 CPU优化实现 75 5.7.2 GPU优化实现 75 5.7.3 内存优化实现 76 5.8 UI系统实现 76 5.8.1 UI系统整体实现思路 76 5.8.2 主菜单UI实现 77 5.8.3 战前准备UI实现 78 5.8.4 战斗UI实现 79 5.8.5 战斗结算UI实现 80 5.8.6 世界地图UI实现 81 5.8.7 星盘UI实现 81 5.8.8 UI系统实现总结 81 5.9 本章小结 82 6 系统测试 83 6.1 测试环境 83 6.1.1 硬件环境 83 6.1.2 软件环境 83 6.2 功能测试 83 6.2.1 测试方法与策略 83 6.2.2 战斗系统功能测试 83 6.2.3 Buff系统功能测试 84 6.2.4 技能系统功能测试 84 6.2.5 AI系统功能测试 85 6.2.6 物品系统功能测试 85 6.2.7 背包系统功能测试 85 6.2.8 拖拽交互功能测试 85 6.2.9 卡牌系统功能测试 86 6.2.10 探索系统功能测试 86 6.2.11 UI系统功能测试 86 6.3 性能测试 87 6.3.1 帧率测试 87 6.3.2 内存占用测试 87 6.3.3 加载时间测试 87 6.3.4 压力测试 87 6.4 兼容性测试 88 6.4.1 操作系统兼容性测试 88 6.4.2 分辨率兼容性测试 88 6.4.3 输入方式兼容性测试 88 6.5 测试结果分析 88 6.5.1 功能测试结果总结 88 6.5.2 性能测试结果总结 89 6.5.3 发现的问题与修复 89 6.5.4 测试结论 89 6.6 本章小结 89 7 总结与展望 91 7.1 工作总结 91 7.1.1 完成的主要工作 91 7.1.2 系统实现成果 91 7.2 主要贡献 92 7.2.1 理论贡献 92 7.2.2 实践贡献 92 7.3 存在的问题 92 7.3.1 系统局限性 92 7.3.2 待改进之处 92 7.4 未来展望 93 7.4.1 功能扩展方向 93 7.4.2 技术优化方向 93 参考文献 95 致谢 99

1 绪论 1.1 研究背景及意义 1.1.1 多元融合游戏类型发展现状与挑战 随着游戏产业的快速发展，单一玩法的游戏已难以满足玩家日益多元化的需求。近年来，融合多种玩法机制的复合型游戏逐渐成为市场主流。肉鸽（Roguelite）游戏以其随机性和高重玩价值深受玩家喜爱；自走棋（Auto Battler）玩法以策略深度和自动战斗机制吸引了大量玩家；搜打撤（Extraction Shooter）玩法以其高风险高回报的探索机制带来了独特的游戏体验；卡牌策略玩法则以丰富的组合可能性提供了深度的策略空间。将上述多种玩法有机融合，已成为游戏设计领域的重要探索方向。以《哈迪斯》《杀戮尖塔》《逃离塔科夫》为代表的融合型游戏，通过将不同玩法机制有机结合，创造出了独特的游戏体验，在全球范围内获得了广泛认可。国内游戏市场同样涌现出大量融合型游戏，如《枪火重生》《元气骑士》等，证明了多玩法融合设计的市场潜力。

然而，多玩法融合游戏的开发面临较大的技术挑战。不同玩法系统之间的交互逻辑复杂，系统架构设计难度高；各系统的数据管理和状态同步需要精心设计；游戏平衡性调整涉及多个系统的协同配合，开发效率有待提高。 1.1.2 游戏开发技术发

高度疑似
AIGC
(99.97%)

高度疑似
AIGC
(99.94%)

高度疑似
AIGC

展趋势与研究意义 随着硬件性能的提升和开发工具的成熟，游戏开发技术呈现出以下发展趋势，这也正是本研究的必要性和意义所在：第一，游戏引擎技术日趋成熟。Unity和Unreal Engine等商业游戏引擎的普及，大大降低了游戏开发的门槛。Unity引擎凭借其跨平台能力、丰富的资源商店和活跃的社区支持，成为中小型游戏团队的首选开发工具[2][3][20]。Hussain等（2020）对Unity引擎进行了系统性技术调查，总结了其在跨平台开发中的技术优势[2]。据统计，全球排名前1000的游戏中，接近50%使用Unity引擎开发[29]。本研究采用成熟的引擎技术和框架，为融合型游戏的系统设计提供了实践验证。

(93.98%)

第二，数据驱动开发模式广泛应用[11][12]。传统的硬编码方式已难以适应快速变化的市场需求，数据驱动开发模式成为主流。Barros等（2018）指出，数据驱动设计是支撑复杂游戏系统的关键方法[11]。通过配置表驱动的方式，策划人员可以独立调整游戏数值，无需修改代码，大大提高了迭代效率。本研究通过配置表驱动的设计方案，验证了数据驱动模式在多系统融合游戏中的有效性。第三，模块化和组件化设计深入人心。随着游戏系统日益复杂，传统的单体架构已难以维护。模块化设计将系统拆分为独立的功能模块，组件化设计将功能封装为可复用的组件，这两种设计思想的结合，使得游戏系统更加灵活、可维护。本研究采用事件驱动架构和模块化设计，为多系统协同工作提供了完整的解决方案。

高度疑似
AIGC
(99.47%)

对比维度 传统OOP设计 模块/组件化设计 扩展性 极差：新增功能需修改继承链，易引发连锁 bug 极强：直接挂载新组件，无需修改原有代码 维护性 极差：深继承链难以调试，牵一发而动全身 优秀：组件独立，单独修改 / 删除不影响其他功能 复用性 受限：仅能通过继承复用，单继承限制大 极高：组件全项目通用，任意对象挂载即可用 开发效率 适合小项目：无需拆分模块，直接写逻辑 适合中大项目：模块化分工，多人并行开发 性能 略优：无组件查找 / 消息分发开销 几乎无差距：Unity 对组件做了极致优化 调试难度 困难：bug 隐藏在继承链中，定位慢 简单：单独禁用组件，快速定位问题

高度疑似
AIGC (97.4%)

学习成本 低：入门简单，符合新手编程思维 中：需要理解「组合思想」，拆分模块 表1.1 传统OOP设计与模块/组件化设计的区别 第四，异步编程技术广泛应用。UniTask等异步编程库的出现[22]，使得游戏中的异步操作更加高效，避免了传统协程的性能问题[16]，提高了代码的可读性和可维护性。Aversa和Dickinson（2019）在Unity优化专著中详细分析了协程的性能瓶颈及异步编程的优势[16]。1.1.3 研究意义与贡献 在上述技术发展趋势的背景下，本研究具有以下意义：在理论贡献上：本文设计了多玩法融合游戏系统架构，将RPG、肉鸽、搜打撤、自走棋、卡牌五种玩法有机融合，为同类游戏的系统设计提供了新的思路和参考。同时，本文采用事件驱动架构、模块化设计和数据驱动开发模式，为多系统协同的游戏架构设计提供了可参考的实践案例。并且本文在开发过程中积累的经验包括性能优化策略、代码组织规范、调试技巧等，也可为同类项目提供參考。

高度疑似
AIGC (92.4%)

在实践意义上：本文详细记录了从需求分析、系统设计到实现测试的完整过程，可为游戏开发初学者提供学习参考。同时，本文采用的技术栈包括Unity引擎、GameFramework框架、UniTask异步编程等，经过实际项目验证，证明了其在多玩法融合游戏开发中的适用性。并且通过该项目的开发，在各个系统模块上产出许多可复用资产，能够在其它项目中进行复用。1.1.4 现有游戏的不足与本研究的目標 分析现有融合型游戏的开发问题，我们确立了本研究的具体目标：（1）系统耦合度高。许多融合型游戏在开发过程中，各玩法系统之间耦合紧密，导致修改一个系统时容易影响其他系统，维护成本高。本研究通过事件驱动架构解耦各系统，实现高内聚低耦合的设计。（2）系统扩展性不足。部分游戏在初期设计时未考虑后续扩展，导致新增功能时需要大量重构工作，不仅增加了开发成本，也影响了游戏的更新节奏。本研究采用模块化设计，为系统扩展预留接口。（3）性能优化不足。融合多种玩法的游戏在战斗场景中需要同时处理大量实体、特效和计算逻辑，对性能提出了更高要求。部分游戏在追求画面效果的同时忽视了性能优化，导致运行卡顿，影响用户体验。本研究重点关注性能优化策略的设计与实现。

高度疑似
AIGC
(99.03%)

（4）开发效率有待提高。传统的开发模式下，策划与程序耦合紧密，策划调整数值需要程序修改代码，这种工作方式效率低下，难以适应快速迭代的需求。本研究采用数据驱动设计，实现策划与程序的分离。综上所述，解决这些问题与完成挑战，是本课题的实践目标与学术贡献所在。1.2 国内外研究现状 1.2.1 游戏市场现状与融合型游戏的成功案例 根据Newzoo全球游戏市场报告，2023年全球游戏市场规模达1844亿美元，其中PC游戏市场规模超过400亿美元。融合多种玩法机制的游戏逐渐占据市场的重要地位，成为推动行业增长的重要力量。国外市场代表作如下：《哈迪斯》（Hades）由Supergiant Games开发，创新性地融合了肉鸽玩法与动作RPG，展现了程序化生成与叙事系统的有机结合[6]。原版游戏销售超过100万份，2024年发售的《哈迪斯II》已销售接近300万份，该系列获得包括TGA年度游戏等多项大奖[30]。

高度疑似
AIGC
(99.85%)

《杀戮尖塔》（Slay the Spire）开创了卡牌肉鸽这一新品类，将卡牌构筑与地牢探索相结合，其数据驱动的卡牌平衡设计成为行业典范[9]。在Steam平台上累计获得207,000条评价，正面评价占96.47%，销售超过400万份，成为独立游戏的标杆[31]。《逃离塔科夫》（Escape from Tarkov）创新性地融合了搜打撤（Extraction Shooter）玩法与RPG成长系统。根据玩家统计数据[32]，月活跃用户达932,020人，日活跃用户约207K+，月收入达\$38.9百万。《云顶之弈》与《刀塔自走棋》开创了全球自走棋热潮，融合MOBA IP与自动战斗机制[8][19]。Xu等（2020）系统研究了自走棋游戏中的阵容挖掘与平衡分析[8]。作为Riot Games推出的自走棋游戏，《云顶之弈》在全球获得了广泛关注[33]。

国内市场代表作如下：根据伽马工委发布的《2023年中国游戏产业报告》[35]，2023年国内游戏市场规模达到3029.64亿元，同比增长13.95%，首次突破3000亿关口。用户规模6.68亿人，为历史新高点。融合型游戏在国内市场的占比逐年提升。《枪火重生》将第一人称射击与肉鸽玩法相结合，全球销量已破250万，成为国内最受欢迎的肉鸽射击游戏[34]。

高度疑似
AIGC (97.7%)

《元气骑士》融合了射击、肉鸽和RPG三种元素，在TapTap累计下载量超过2166万次，成为国内同类型游戏的标杆。1.2.2 融合型游戏的设计特点与启示 通过分析上述代表作，我们总结出融合型游戏设计的几个关键特点：（1）多系统协同与调性统一：融合型游戏的核心在于将不同玩法系统有机整合，保持整体游戏调性的一致性。《哈迪斯》通过精细的叙事和视觉风格保持肉鸽与动作RPG的统一体验。

高度疑似
AIGC
(99.99%)

（2）数据驱动的平衡设计：配置表驱动已成为支撑复杂游戏系统的关键技术[11][12]。《杀戮尖塔》超过200张卡牌的平衡、《云顶之弈》每赛季的机制调整，都依赖于高效的数据驱动设计[9]。Pfau和Seif El-Nasr（2024）提出了结合玩家驱动和数据驱动的游戏平衡分析方法[12]。（3）永久进度与重玩价值：融合型游戏普遍采用Meta-progression等机制，确保玩家失败也能获得进度感，从而提升长期参与度。（4）版本化迭代与快速响应：通过配置变更实现快速版本迭代，保持玩法活力，这对融合型游戏尤为重要，因为多系统的平衡调整更加复杂。1.2.3 游戏开发技术与框架现状 在技术方面，根据2024年游戏技术现状调查报告[29]，Unity因其跨平台能力、丰富的资源商店和活跃的开发社区[2][20]，已成为游戏开

发的主流选择。最新数据显示，Unity在2024年的市场份额为47%，在Steam游戏中占51%，在手机游戏中更是高达73%[29]。

GameFramework等开源框架[24]通过提供完整的系统架构（如流程管理、事件系统、资源管理等），被数百个商业项目采用。Sripada等（2016）提出了可扩展游戏框架的架构设计方法[5]。UniTask等异步编程库[22]已成为行业标准，下载量超过100万次。数据驱动开发模式的应用已成为行业共识[11][12]。根据GDC 2023游戏开发者大会的相关报告，游戏开发团队普遍采用数据驱动的开发模式。Excel配置表结合代码生成工具（如DataTableGenerator）的方案因其低成本、易上手的特点，在中小型团队中应用最为广泛。1.2.4 学术研究现状与本研究的定位 虽然游戏系统设计、游戏AI、游戏平衡性方向已有较完整的学术研究成果，但融合多种玩法的游戏系统设计的学术研究相对较少。学术界对单一游戏类型（如肉鸽、卡牌、自走棋）的研究较为深入，但对融合型游戏系统的完整设计方案的学术论文仍然稀缺。大多数融合型游戏的设计经验仍停留在业界经验总结阶段，缺乏系统的学术研究和理论提升。特别是在以下方面，现有研究存在明显空白：

(1) 多系统间的耦合控制：如何在保持系统独立性的同时实现高效协同 (2) 跨系统平衡设计：融合多玩法后的系统平衡策略和方法 (3) 配置驱动架构：在复杂融合游戏中的设计与实现最佳实践 (4) 融合型游戏的性能优化：多系统并行工作下的性能瓶颈识别与优化策略 基于上述分析，本研究的定位为：以Clash of Gods多元素融合游戏为实践案例，系统研究融合型游戏的架构设计方法、系统协同机制、数据驱动设计与性能优化策略。 本研究通过系统的架构设计和实践验证，为融合型游戏的开发提供了完整的解决方案，填补了学术研究与产业实践之间的空白，对于推动多玩法融合游戏的发展具有重要意义。

1.3 主要研究内容 1.3.1 研究内容 本文的主要研究内容包括：(1) 多玩法融合游戏系统架构设计。研究如何设计一个可扩展、可维护的游戏系统架构，支持RPG、肉鸽、搜打撤、自走棋、卡牌等多种玩法的协同工作，包括分层架构设计、模块划分、事件系统设计等。(2) 核心游戏系统设计与实现。包括战斗系统、物品背包系统、策略卡系统、探索系统等核心模块的设计与实现。(3) 数据驱动开发模式应用。研究如何通过配置表驱动游戏数据，实现策划与程序的分离，提高开发效率。

(4) 性能优化策略研究。研究游戏性能优化的方法，包括CPU优化、GPU优化、内存优化等，确保游戏在PC端的流畅运行。 1.3.2 研究方法 本文采用的研究方法包括：(1) 文献研究法。通过查阅游戏开发相关书籍、论文和技术文档，了解游戏开发的理论基础和技术现状。(2) 案例分析法。分析市场上成功的融合型游戏，总结其设计特点和成功经验，为本项目提供参考。(3) 实践验证法。通过实际开发项目，验证技术方案的可行性和有效性，在实践中发现和解决问题。(4) 测试评估法。通过功能测试、性能测试和兼容性测试，评估系统的质量和性能，确保达到预期目标。 1.3.3 技术路线 本文的技术路线分为四个阶段：

(1) 技术选型与框架搭建。选择Unity引擎作为开发工具，采用GameFramework框架作为底层架构，搭建项目基础框架，确定技术栈和开发规范。(2) 核心系统开发。按照模块化设计思想，依次开发战斗系统、物品背包系统、策略卡系统、探索系统等核心模块，每个模块独立开发、独立测试。(3) 系统集成与测试。将各模块集成到一起，进行功能测试、性能测试和兼容性测试，确保各模块协同工作正常。(4) 优化与完善。根据测试结果进行性能优化，修复发现的问题，完善系统功能，撰写论文。 1.4 本文的组织结构 本文共分为七章，各章内容安排如下：第1章为绪论，介绍课题研究背景、研究意义、国内外现状、主要研究内容与方法以及本文的组织结构。

第2章为相关技术，介绍Unity游戏引擎、GameFramework框架、多玩法融合游戏设计、系统架构设计模式和配置表驱动设计等相关技术知识。第3章为需求分析，分析游戏的功能需求和非功能需求，进行用例分析和可行性分析。第4章为系统设计，详细设计系统总体架构、战斗系统、物品与背包系统、策略卡系统、探索系统、数据表与配置系统以及UI系统。第5章为系统实现，介绍各核心系统的具体实现过程，包括技术美术系统、战斗系统、物品系统、卡牌系统、探索系统、数据配置系统的实现以及性能优化策略。第6章为系统测试，介绍测试环境、功能测试、性能测试、兼容性测试和测试结果分析。

第7章为总结与展望，总结本文的主要工作和贡献，指出存在的问题，展望未来研究方向。 2 相关技术 本章介绍论文涉及的相关技术，包括Unity游戏引擎、GameFramework框架、多玩法融合游戏设计、系统架构设计模式和配置表驱动设计等内容，为后续章节的系统设计与实现奠定技术基础。 2.1 Unity游戏引擎 2.1.1 游戏引擎概述 游戏引擎是为游戏开发者提供的一套软件开发框架，集成了图形渲染、物理模拟、音频处理、输入管理、网络通信等核心功能。使用游戏引擎可以大大提高游戏开发效率，使开发者能够专注于游戏逻辑和内容创作，而无需从零开始构建底层系统。目前主流的游戏引擎包括Unity、Unreal Engine、Godot、Cocos2d-x等。其中，Unity引擎凭借其跨平台能力、易用性和丰富的生态系统，成为全球使用最广泛的游戏引擎之一[2][3][20]。Hussain等（2020）对Unity引擎进行了系统性技术调查，总结了其在跨平台开发、渲染管线、物理引擎等方面的技术优势[2]。

2.1.2 Unity引擎特点 Unity引擎具有以下显著特点：具有强大跨平台能力：Unity支持一次开发、多平台发布，可发布到Windows、macOS、Linux、iOS、Android、WebGL、PlayStation、Xbox、Switch等主流平台。使用组件化设计思想：Unity采用组件-游戏对象（Component-GameObject）的设计模式，功能模块高度解耦，便于复用和扩展。拥有丰富的资源商店：Unity Asset Store提供了海量的插件、模型、材质、音效等资源，开发者可以快速获取所需资源，加速开发进程。 2.1.3 Unity核心概念 Unity引擎的核心概念包括：游戏对象（GameObject）是Unity场景中的基本实体，可以理解游戏中的各种物体，如角色、敌人、道具、摄像机等。游戏对象本身不包含具体功能，需要通过挂载组件来获得各种能力。

组件（Component）是实现具体功能的模块，如Transform组件控制位置、旋转和缩放，MeshRenderer组件负责渲染模型，Collider组件处理碰撞检测等。Unity内置了大量常用组件，开发者也可以编写自定义组件。预制体（Prefab）是一种资源类型，用于存储可复用的游戏对象及其组件配置。通过预制体可以快速创建多个相同的游戏对象，修改预制体会自动同步到所有实例。场景（Scene）是游戏内容的容器，包含当前关卡或界面的所有游戏对象。一个游戏通常由多个场景组成，如主菜单场景、游戏场景、战斗场景等。资源（Asset）是Unity项目中的各种素材，包括模型、材质、贴图、音频、动画、脚本等。资源存储在Assets目录下，通过AssetBundle可以实现资源的动态加载和版本管理。

2.2 GameFramework框架 2.2.1 框架概述 GameFramework是一个基于Unity的开源游戏开发框架[24]，采用模块化设计，提供事件系统、实体系统、UI系统、数据表系统等基础设施[5]。Sripada等（2016）研究了可扩展框架的架构设计

高度疑似
AIGC
(95.79%)

高度疑似
AIGC
(99.72%)

高度疑似
AIGC
(85.18%)

高度疑似
AIGC
(98.75%)

高度疑似
AIGC
(90.46%)

轻度疑似
AIGC
(65.13%)

中度疑似
AIGC
(77.72%)

原则，强调模块化和事件驱动架构的重要性[5]。2.2.2 核心模块 GameFramework框架包含以下核心模块：事件系统（Event）：提供了观察者模式的实现，用于模块间的解耦通信。模块可以订阅感兴趣的事件，当事件触发时收到通知。事件系统支持立即触发和延迟触发两种模式。 实体系统（Entity）：用于管理游戏中的实体对象，如角色、敌人、特效等。实体系统支持实体的显示、隐藏、回收和对象池管理，可以有效减少实例化和销毁带来的性能开销。 UI系统（UI）：提供了完整的UI管理方案，支持UI界面的打开、关闭、动画、层级管理等功能。UI系统与数据表系统集成，可以通过配置表管理UI信息。

数据表系统（DataTable）：用于管理游戏配置数据，支持从二进制文件或文本文件加载数据。数据表系统与Excel工具链集成，策划人员可以通过Excel编辑数据，自动生成代码和数据文件。 网络系统（Network）：提供了网络通信功能，支持TCP和UDP协议，使用Protobuf进行序列化。网络系统封装了连接管理、心跳检测、断线重连等常用功能。 资源系统（Resource）：负责资源的加载和管理，支持同步加载和异步加载。资源系统与AssetBundle集成，支持资源版本管理。 流程系统（Procedure）：基于有限状态机（FSM）实现，用于管理游戏的整体流程，如启动流程、登录流程、游戏流程等。每个流程是一个独立的状态，流程之间可以切换。

2.2.3 框架优势 GameFramework框架具有以下优势： 第一，模块化设计。框架的各个模块相互独立，开发者可以根据需要选择使用哪些模块，不会产生强耦合。 第二，完善的文档和示例。框架提供了详细的API文档和丰富的示例项目，开发者可以快速上手。 第三，活跃的社区支持。框架在国内Unity开发者社区有较高的知名度，遇到问题可以快速获得帮助。 第四，性能优化。框架在实现时考虑了性能因素，如对象池、异步加载、事件批处理等，可以有效提高游戏性能。 2.3 多玩法融合游戏设计 2.3.1 主要玩法类型介绍 Clash of Gods融合了五种主要玩法机制，各玩法的特点如下： RPG（角色扮演游戏）玩法为游戏提供了角色成长和叙事框架。玩家扮演召唤师，通过探索、战斗、收集物品不断强化自身实力。RPG玩法的核心在于角色属性系统、装备系统和技能系统的设计，为玩家提供长期的成长目标。

肉鸽（Roguelite）玩法为游戏提供了随机性和重玩价值[6][7][18]。每次游戏流程中，地图布局、敌人分布、物品掉落等元素具有一定的随机性，使得每次游戏体验都有所不同。肉鸽玩法的核心在于随机性与策略性的平衡[18]，以及死亡惩罚与永久成长的设计。 Gellel和Sweetser（2020）提出了混合式程序化生成方法，结合规则驱动与数据驱动的生成策略[7]。 搜打撤（Extraction）玩法为游戏提供了探索张力。玩家在探索场景中自由移动，搜集物品和资源，同时需要应对敌人的威胁，并在适当时机撤离。搜打撤玩法的核心在于风险与收益的权衡，以及探索过程中的紧张感。 自走棋（Auto Battler）玩法为游戏提供了策略深度[8][19]。战斗采用自动战斗机制，玩家在战前布置棋子阵容，战斗自动进行。自走棋玩法的核心在于棋子组合策略、阵容搭配和资源管理[8]。 Xu等（2020）研究了自走棋游戏中的阵容挖掘与平衡分析方法[8]。

卡牌策略玩法为游戏提供了额外的策略层次。玩家通过净化敌人获得策略卡牌，在战斗中使用卡牌改变战局。卡牌玩法的核心在于卡牌效果的多样性和使用时机的判断。 2.3.2 多玩法融合设计要素 多玩法融合游戏的核心设计要素包括：玩法协同设计是融合游戏的关键。各玩法系统之间需要有机联系，而非简单叠加。在Clash of Gods中，探索（搜打撤）触发战斗（自走棋），战斗胜利获得物品（RPG背包）和卡牌（卡牌策略），物品和卡牌强化战斗能力，形成完整的玩法循环。系统解耦设计保证了各玩法系统的独立性。通过事件驱动架构，各系统之间通过事件通信而非直接调用，降低了系统耦合度，便于独立开发和维护。

数据驱动设计支持快速迭代[11][12]。各玩法系统的参数通过配置表管理，策划人员可以独立调整各系统的数值，无需修改代码，大大提高了迭代效率。 Barros等（2018）指出，数据驱动设计是支撑复杂游戏系统的关键方法，能够实现游戏内容的最大化设计[11]。 2.3.3 多玩法融合游戏优势 多玩法融合游戏具有以下优势： 第一，玩法多样性高。融合多种玩法机制，为玩家提供了丰富的游戏体验，避免了单一玩法带来的审美疲劳。 第二，重玩价值高。肉鸽元素的引入使得每次游戏体验都有所不同，大大提高了游戏的重玩价值。 第三，策略深度强。自走棋和卡牌玩法的结合，为玩家提供了丰富的策略选择空间，满足了核心玩家对策略深度的需求。

第四，探索乐趣足。搜打撤玩法带来了独特的探索张力，使玩家在探索过程中保持高度的专注和紧张感。 2.4 系统架构设计模式 2.4.1 事件驱动架构 事件驱动架构是实现模块解耦的核心机制[5][28]。通过发布-订阅模式，组件之间通过事件进行通信，而不是直接调用，从而降低系统耦合度。 Nystrom在《游戏编程模式》中系统阐述了事件系统的设计原则和实现方法[28]。在游戏开发中，事件驱动架构广泛应用于模块间通信，如战斗系统通知UI系统更新显示、物品系统通知背包系统刷新列表等。 2.4.2 MVC模式 MVC（Model-View-Controller）是一种经典的软件设计模式[4][28]，将系统分为三个部分：模型（Model）负责数据管理，视图（View）负责界面显示，控制器（Controller）负责业务逻辑。在游戏开发中，MVC模式帮助实现UI与逻辑的分离，提高了代码的可维护性和可测试性。 Maharjan（2018）在Android游戏开发中验证了MVVM架构（MVC的扩展）的有效性[4]。在游戏开发中，MVC模式的应用如下：模型层管理游戏数据，如角色属性、物品数据等；视图层负责UI显示和特效渲染；控制器层处理用户输入和游戏逻辑。 MVC模式的优点包括职责分离、代码复用、便于测试等。在Unity开发中，通常将UI作为视图层，脚本作为控制器层，数据类作为模型层。 2.4.3 模块化设计 模块化设计是将系统划分为独立功能模块的设计方法。每个模块负责特定的功能，模块之间通过定义良好的接口进行通信。

模块化设计的优点包括： 第一，降低复杂度。将复杂系统分解为简单模块，每个模块职责单一，便于理解和开发。 第二，提高复用性。独立的功能模块可以在不同项目中复用。 第三，便于协作。不同开发者可以并行开发不同模块，提高开发效率。 第四，易于维护。模块独立，修改一个模块不会影响到其他模块。 在游戏开发中，常见的模块划分包括战斗模块、物品模块、任务模块、社交模块等。 2.4.4 组件化设计 组件化设计是将功能封装为可复用组件的设计方法。与模块化设计相比，组件化设计更强调功能的可组合性。 Unity引擎本身就是组件化设计的典型代表。游戏对象通过挂载不同组件获得不同功能，如移动组件、攻击组件、AI组件等。这种设计使得功能可以灵活组合，提高了代码复用率。

组件化设计的优点包括灵活性高、可复用性强、易于扩展等。在游戏开发中，应优先考虑组件化设计，将功能封装为独立组件。 2.5 配置表驱动设计 2.5.1 配置表概念 配置表驱动设计是一种将游戏数据与代码分离的设计方法。游戏数据存储在配置表中，程序通过读取配置表获取数据，而不是将数据硬编码在代码中。配置表通常使用Excel或类似的表格工具编辑，每行代表一条数据记录，每列代表一个数据字段。配置表可以存储各种游戏数据，如角色属性、技能参数、物品信息、关卡配置等。 2.5.2 配置表优势 配置表驱动设计具有以下优势： 第一，策划友好。策划人员可以通过Excel编辑游戏数据，无需

中度疑似
AIGC
(70.56%)

中度疑似
AIGC (73.1%)

中度疑似
AIGC (79.8%)

高度疑似
AIGC
(90.45%)

高度疑似
AIGC
(91.48%)

轻度疑似
AIGC
(56.21%)

高度疑似
AIGC
(96.26%)

程序介入，大大提高了迭代效率。

第二，数据与代码分离。数据修改不需要重新编译代码，降低了发布风险。第三，便于版本管理。配置表可以独立版本管理，支持数据回滚和差异对比。第四，便于测试。测试人员可以独立修改配置进行测试，无需重新打包。

2.5.3 配置表设计原则

配置表设计应遵循以下原则：第一，单一职责。每张表只存储一类数据，避免数据冗余和耦合。第二，字段规范。字段命名应清晰明确，类型应合理选择，避免使用过于复杂的嵌套结构。第三，注释完善。每张表应有清晰的注释说明，每个字段应有描述信息。第四，ID设计合理。ID应具有唯一性和可读性，可以采用分段设计，如10000-19999为消耗品，20000-29999为任务道具等。

第五，引用关系清晰。表与表之间的引用关系应明确，避免循环引用。

2.6 本章小结

本章介绍了论文涉及的相关技术，包括Unity游戏引擎的特点和核心概念、GameFramework框架的模块和优势、多玩法融合游戏的设计要素、系统架构设计模式以及配置表驱动设计方法。这些技术知识为后续章节的系统设计与实现提供了理论基础。Unity引擎作为开发工具，提供了跨平台能力和丰富的开发功能。GameFramework框架作为底层架构，提供了事件系统、实体系统、UI系统、数据表系统等核心模块。多玩法融合游戏设计理论指导了游戏系统的设计方向。事件驱动架构、MVC模式、模块化设计和组件化设计为系统架构设计提供了方法论。配置表驱动设计实现了数据与代码的分离，提高了开发效率。

3 需求分析

本章对Clash of Gods游戏系统进行全面的需求分析，包括业务需求、用户需求、功能需求分析、非功能需求分析、用例分析和可行性分析，为后续系统设计提供依据。

3.1 业务需求

随着游戏产业的快速发展，多玩法融合游戏已成为市场主流。然而，现有的多玩法融合游戏开发面临着一系列亟待解决的问题。在系统架构层面，融合型游戏由于需要整合多个不同的玩法系统，导致系统之间的交互逻辑异常复杂。不同的玩法系统往往存在紧密耦合，修改一个系统时容易影响其他系统，使得整个项目的维护难度大幅提升。这种架构问题不仅增加了开发成本，也严重限制了游戏的后续扩展空间。

在开发效率方面，传统的硬编码方式导致策划和程序之间存在紧密耦合。每当策划需要调整游戏数值时，都需要程序员修改代码并重新编译打包，这个过程不仅耗时，而且容易引入bug。这种低效的开发流程使得游戏无法快速响应市场反馈和玩家需求，大大降低了迭代效率。在系统扩展性方面，部分现有游戏在初期设计时未充分考虑后续扩展的可能性，导致当需要新增功能或修改现有系统时，往往需要进行大量的重构工作。这不仅增加了开发成本，也打乱了既定的开发计划，影响游戏的更新节奏。在性能优化方面，融合多种玩法的游戏在战斗场景中需要同时处理大量实体、特效和计算逻辑。这对游戏的性能提出了更高要求，许多游戏在追求画面效果的同时忽视了性能优化，导致运行卡顿，严重影响用户体验。

基于上述现状，业界迫切需要一种新型的游戏系统架构设计方案。这种方案应当采用事件驱动架构实现系统间的有效解耦，运用模块化设计支持灵活的系统组合，采用数据驱动开发模式将游戏数据与代码分离，从而提高开发效率、支持灵活扩展，并通过系统的性能优化技术确保游戏的流畅运行。

3.2 用户需求

本系统的目标用户是喜欢策略、探索和自走棋玩法的核心向玩家。这类玩家具有较强的游戏理解能力和策略思维，他们不满足于简单的娱乐体验，而是追求在游戏中获得深度的参与感和成就感。这些核心玩家最为关注的是玩法的多样性。单一的游戏玩法容易导致审美疲劳，无法长期吸引玩家。通过融合RPG的角色成长系统、肉鸽的随机探索机制、搜打撤的风险收益平衡、自走棋的策略深度以及卡牌系统的组合可能性，游戏能够为玩家提供多维度的游戏体验，使每次游玩都能产生不同的感受。

同时，玩家期望在探索的过程中体验到紧张感。搜打撤玩法中的高风险高回报机制，使得每一次的探索决策都变成一次风险与收益的权衡。玩家需要在获取更多资源的诱惑与生命安全之间做出取舍，这种紧张的决策过程能够大幅增强游戏的沉浸度。此外，核心玩家对游戏的策略深度有着明确的需求。自走棋玩法提供的棋子搭配、羁绊选择等策略维度，以及卡牌系统提供的丰富的卡牌组合空间，为玩家的每一个决策赋予了意义。玩家可以通过深思熟虑的策略选择来应对不同的游戏局面，这种策略性的游戏过程满足了他们对智力挑战的追求。最重要的是，玩家需要持久的成长满足感。RPG玩法提供的角色等级提升、装备获取等明确的成长目标，使得长期游玩能够带来持续的进度反馈。而肉鸽玩法的永久升级机制确保了即使游戏失败，玩家也能获得某种形式的进度和收获，从而激励他们进行下一轮游玩。这种既有短期的紧张刺激，又有长期的成长目标的设计，能够有效地维持玩家的长期参与度和游戏粘性。

3.3 功能需求分析

3.3.1 游戏概述

Clash of Gods是一款以世界神话为背景的多玩法融合游戏，目标平台为PC端

(Windows)。游戏融合了RPG、肉鸽 (Roguelite)、搜打撤 (Extraction)、自走棋 (Auto Battler) 和卡牌策略五种玩法机制。玩家扮演召唤师，在探索地图中搜集物品、与敌人遭遇并触发战斗。游戏功能可按四大主要系统进行分解 (如图3.1所示)，各系统共包含50项功能需求。图 3.1 功能需求分解树 具体分解如下： (1) 探索系统 (9项需求)：负责游戏基础探索和敌人交互。包括自由移动、敌人视野检测、敌人AI状态机、三种触发方式、精英敌人广播、净化系统、战后隐身、污染值系统等功能。

(2) 战斗系统 (10项需求)：负责游戏核心战斗机制。包括棋子部署、自动战斗AI、技能系统、Buff/Debuff、伤害计算、命中检测、先手效果、脱战机制、战斗结算、场景过渡等功能。 (3) 物品系统 (9项需求)：负责物品和背包管理。包括物品分类、背包管理、拖拽交互、装备系统、词条系统、羁绊系统等功能。 (4) 卡牌系统 (5项需求)：提供额外策略层次。包括卡牌获取、手牌管理、卡牌使用、效果多样性、卡牌池管理等功能。接下来分别介绍各系统的详细需求。

3.3.2 探索功能需求

探索系统是游戏的基础玩法，主要功能需求整理如表3.1所示：

编号	需求名称	需求描述
EXP-01	自由移动	玩家通过键鼠在三维场景中自由移动，支持多种视角模式

EXP-02	敌人视野检测	敌人具有扇形视觉锥和圆形感知范围，玩家进入后触发警戒
EXP-03	敌人AI状态机	敌人拥有巡逻、警戒、追击、休息、战斗等状态，状态间自动切换
EXP-04	偷袭触发	玩家从敌人背后接近且敌人未警觉时，显示偷袭提示
EXP-05	遭遇战触发	玩家正面接近未警觉敌人时，显示遭遇战提示
EXP-06	敌方先手触发	敌人追上玩家时自动触发战斗，敌方获得先手效果
EXP-07	精英敌人广播	精英敌人发现玩家后广播位置，召集范围内其他敌人
EXP-08	净化系统	击败特殊敌人后可净化，获得对应策略卡牌
EXP-09	战后隐身	战斗结束后玩家进入隐身状态，敌人暂时无法检测
EXP-10	污染值系统	玩家污染值随时间增长，战斗失败额外增加，满值游戏结束

表3.1	探索系统功能需求表	
3.3.3 战斗功能需求	战斗系统是游戏的核心玩法，主要功能需求如表3.2所示：	
编号	需求名称	需求描述
CMB-01	棋子部署	战前在战场格子选择并部署我方棋子
CMB-02	自动战斗AI	战斗开始后棋子自动寻敌、移动、攻击
CMB-03	技能系统	棋子拥有普攻和技能，技能有冷却、范围、效果等参数
CMB-04	Buff/Debuff系统	支持攻击力增减、减速、灼烧、护盾等多种效果，可叠层
CMB-05	伤害计算	支持物理/魔法/真实三类伤害，考虑攻防暴击等属性

中度疑似
AIGC
(73.09%)

高度疑似
AIGC
(99.92%)

高度疑似
AIGC
(86.69%)

高度疑似
AIGC
(99.24%)

高度疑似
AIGC
(95.24%)

高度疑似
AIGC
(99.91%)

高度疑似
AIGC
(91.48%)

高度疑似
AIGC
(99.64%)

CMB-06 命中检测 支持范围检测、射线检测、弹道检测三种方式 CMB-07 先手效果 根据战斗触发类型，给予不同先手Buff CMB-08 脱战机制 战斗中可尝试脱战，成功率随回合数递增，失败扣血

CMB-09 战斗结算 胜利：销毁敌人、发放奖励；失败：增加污染值 CMB-10 场景过渡 战斗准备/结束时播放溶解过渡，摄像机隔离敌人 表3.2 战斗系统功能需求表 3.3.4 物品管理需求 物品背包系统管理游戏中的各种道具，主要功能需求如表3.3所示： 编号 需求名称 需求描述 ITM-01 物品分类 支持消耗品、任务道具、宝物、装备四类物品 ITM-02 背包容量 背包固定容量，支持添加/删除/排序操作 ITM-03 堆叠合并 可堆叠物品自动合并，超过最大堆叠数时拆分 ITM-04 物品使用 消耗品使用后触发效果（回血、获得金币等） ITM-05 装备系统 装备/宝物可装备到对应槽位，卸装后归还背包 ITM-06 拖拽交互 通过拖拽实现物品移动、装备、丢弃等操作

ITM-07 物品详情 点击物品查看详细属性、描述和词条信息 ITM-08 词条系统 宝物具有随机词条，提供额外属性加成 ITM-09 羁绊系统 特定宝物组合激活羁绊，提供额外加成或特殊效果 表3.3 物品背包系统功能需求表 3.3.5 卡牌策略需求 策略卡系统为战斗提供额外的策略层次，主要功能需求如表3.4所示： 编号 需求名称 需求描述 CRD-01 卡牌获取 通过净化敌人或完成任务获得策略卡牌 CRD-02 手牌管理 战斗中维护手牌列表，支持查看和选择 CRD-03 卡牌使用 在战斗中使用卡牌触发效果，消耗手牌 CRD-04 效果多样性 卡牌效果包括棋子强化、敌方削弱、战场改变等 CRD-05 卡牌池管理 系统维护卡牌池，控制获取概率和总量

表3.4 策略卡系统功能需求表 3.4 非功能需求分析 非功能需求对于确保系统满足用户和业务目标至关重要。本系统不仅需要关注功能实现，还需要关注系统实现的可行性、易用性、性能等方面。非功能需求汇总如表3.5所示： 类别 编号 需求描述 量化指标 性能 NFR-01 帧率稳定 PC端稳定60FPS 性能 NFR-02 内存控制 运行时不超过500MB 性能 NFR-03 加载时间 场景切换≤5秒，战斗加载≤3秒 性能 NFR-04 响应时间 UI交互响应≤100毫秒 可维护 NFR-05 模块解耦 模块间通过事件系统通信，无直接依赖 可维护 NFR-06 配置分离 游戏数据通过DataTable管理，策划可独立修改 可扩展 NFR-07 内容扩展 新增棋子/物品/卡牌仅需配置表+效果类 兼容 NFR-08 平台兼容 Windows 10及以上 兼容 NFR-09 分辨率适配 支持1080p/1440p/4K 表3.5 非功能需求汇总表 3.5 用例分析 3.5.1 参与者识别 系统的主要参与者为玩家（Player），玩家游戏的唯一用户，通过键鼠与游戏交互。 3.5.2 用例描述 根据功能需求分析，系统的主要用例整理如表3.6： 用例编号 用例名称 参与者 前置条件 基本流程 异常流程 UC-01 探索地图 玩家 进入探索状态 1.玩家通过键鼠移动 2.场景加载敌人和道具 3.玩家自由探索各区域 切换场景时加载失败→提示错误 UC-02 触发战斗 玩家 探索中发现敌人 1.检测战斗触发类型 2.判断先手效果 3.进入战斗准备 4.玩家部署棋子 5.确认开始战斗 敌人数据缺失→跳过战斗 UC-03 自动战斗 系统 战斗准备完成 1.棋子AI寻敌 2.移动到攻击范围 3.执行普攻/技能 4.计算伤害和Buff 5.判定胜负 所有棋子阵亡→战斗失败 UC-04 使用策略卡 玩家 战斗进行中 1.打开手牌面板 2.选择卡牌 3.选择目标 4.执行卡牌效果 5.消耗手牌 无有效目标→提示无法使用 UC-05 管理背包 玩家 探索或战斗准备中 1.打开背包UI 2.拖拽物品移动/装备 3.点击查看详情 4.使用消耗品 背包已满→提示空间不足 UC-06 脱战 玩家 战斗进行中 1.点击脱战按钮 2.系统计算成功率 3.成功→退出战斗 失败→扣除生命值，继续战斗 UC-07 偷袭敌人 玩家 接近未警觉敌人背后 1.显示偷袭提示 2.按空格触发 3.选择偷袭Debuff 4.施加给敌方后进入战斗 敌人转身发现→变为遭遇战 表3.6 关键用例描述表 本系统的外部交互以玩家（Player）为核心触发源，主要围绕探索、战斗、物品与卡牌四个子系统展开：玩家在探索阶段移动与遭遇敌人并触发/脱离战斗，通过净化等机制获得收益；进入战斗后进行战前部署并在自动战斗过程中使用策略手段干预战局；同时通过背包/仓库与装备、消耗品等管理资源，并在战斗中使用策略卡形成额外的策略层次。系统总体用例如图3.2所示： 图 3.2 系统总体用例图 探索系统负责大地图探索与遭遇逻辑，玩家主要通过移动探索、敌人感知与战斗触发（偷袭/遭遇/被追击）、净化敌人以及脱战等方式与世界交互。探索系统用例如图3.3所示： 图 3.3 探索系统用例图 战斗系统承载核心战斗流程，玩家在战前进行棋子部署，战斗中以自动战斗为主并可通过策略卡等方式进行干预，同时包含尝试脱战/逃脱与战后结算等关键环节。战斗系统用例如图3.4所示： 图 3.4 战斗系统用例图 物品系统负责资源与道具管理，玩家通过物品拾取与获取、背包/仓库管理、装备穿戴与卸下、消耗品使用、物品详情查看及拖拽交互等用例完成资源配置与角色强化。物品系统用例如图3.5所示： 图 3.5 物品系统用例图 卡牌系统为战斗提供策略增量，玩家通过卡牌获取/解锁、手牌查看与管理、卡牌使用（目标选择与范围预览）以及效果结算等用例参与战斗策略。卡牌系统用例如图3.6所示： 图 3.6 卡牌系统用例图 此外，敌人AI巡逻、视野检测、广播、污染值增长等属于系统内部自动机制，将在对应子系统的设计与流程章节中结合实现进行说明。 3.6 可行性分析 3.6.1 技术可行性 本项目采用成熟的技术栈，具有充分的技术支持：（1）Unity引擎在游戏开发中广泛应用，相关资源丰富（2）GameFramework框架提供了完善的基础设施（3）UniTask和DOTween等库提供了高效的工具支持 3.6.2 经济可行性 本项目为毕业设计项目，主要成本为开发人员的时间成本。开发工具方面，Unity引擎个人版免费，GameFramework框架开源免费，主要开发工具无需额外费用。美术资源方面，可以使用Unity Asset Store的免费资源和自制资源，控制美术成本。服务器方面，本项目为单机游戏，无需服务器费用。 综上，本项目在经济层面具有可行性。 3.6.3 时间可行性 本项目的开发周期约为6个月，按照以下阶段规划：第一阶段（1个月）：需求分析和系统设计，完成需求文档和设计文档。第二阶段（3个月）：核心系统开发，包括战斗系统、物品系统、卡牌系统、探索系统等。第三阶段（1个月）：系统集成和测试，修复发现的问题。第四阶段（1个月）：优化完善，撰写论文。按照此计划，在6个月内完成项目开发和论文撰写是可行的。 3.7 本章小结 本章对Clash of Gods游戏系统进行了全面的需求分析。在业务需求和用户需求的基础上，详细分析了功能需求，包括探索系统、战斗系统、物品背包系统和策略卡系统的具体功能需求，共计42个具体需求项。在非功能需求方面，明确了性能、可维护性、可扩展性和兼容性等方面的量化要求。通过用例分析，识别了系统的主要参与者和用例，绘制了用例图并描述了关键用例的详细流程。通过可行性分析，从技术、经济和时间三个维度论证了项目的可行性。本章的需求分析结果将作为后续系统设计的依据，确保系统设计能够满足用户需求和技

高度疑似
AIGC
(93.99%)

高度疑似
AIGC
(97.06%)

4.1 系统总体架构设计 4.1.1 分层架构设计 系统采用四层架构设计，从下到上依次为基础框架层、核心系统层、业务逻辑层和表现层，如图4.1所示： 图 4.1 系统分层架构图 各层之间单向依赖，上层依赖下层，下层不感知上层，保证了系统的低耦合性。基础框架层是整个系统的基石，包括Unity引擎和GameFramework框架。Unity引擎提供渲染管线、物理引擎、音频系统、输入系统等底层能力；GameFramework框架在Unity之上提供了游戏开发所需的基础设施，包括资源管理、对象池、有限状态机等通用功能。核心系统层基于GameFramework框架构建，包含事件系统（Event）、实体系统

(Entity)、数据表系统 (DataTable)、UI系统 (UI)、资源系统 (Resource)、流程系统 (Procedure) 等核心模块。这些模块为上层业务逻辑提供标准化的服务接口，屏蔽了底层实现细节。业务逻辑层是游戏功能的核心，包含战斗系统、物品系统、卡牌系统、探索系统、配置系统等游戏特有的业务模块。各业务模块通过核心系统层提供的事件系统进行解耦通信，通过数据表系统读取配置数据。表现层负责将游戏状态以视觉和听觉的形式呈现给玩家，包括UI界面、粒子特效、角色动画、音效等。表现层通过订阅业务逻辑层发出的事件来更新显示状态，实现了逻辑与表现的分离。

4.1.2 系统整体架构图 系统整体架构如图4.2所示: 图 4.2 系统整体架构图 游戏流程由ProcedureManager通过有限状态机管理，控制游戏在LaunchProcedure (启动)、PreloadProcedure (预加载)、MenuProcedure (主菜单)、ExploreProcedure (探索)、CombatProcedure (战斗) 等流程之间切换。各业务系统通过GameFramework事件系统进行解耦通信。例如，探索系统中的CombatTriggerManager触发战斗时，通过发布CombatStartEventArgs事件通知战斗系统；战斗系统结束后发布CombatEndEventArgs事件，通知探索系统和物品系统进行后续处理。数据流向遵循单向原则：配置数据从DataTable流向各业务系统，运行时数据存储在DataModel中，UI通过订阅事件获取数据更新。

4.1.3 核心系统模块划分 系统按功能职责划分为以下核心模块，各模块的职责边界清晰：战斗模块 (Combat)：职责范围包括战斗流程管理、棋子实体管理、AI决策、技能执行、Buff效果管理、伤害计算、命中检测等。对外提供战斗开始/结束接口，通过事件系统对外广播战斗状态变化。探索模块 (Explore)：职责范围包括探索场景中的敌人实体管理、敌人AI状态机、视野检测、战斗触发检测、净化系统等。与战斗模块通过事件系统协作，负责战斗的发起和结束后的场景恢复。物品模块 (Item)：职责范围包括物品数据管理、背包容器管理、装备系统、宝物词条系统、羁绊系统、物品效果执行等。提供统一的物品操作接口，通过事件系统通知UI更新。卡牌模块 (Card)：职责范围包括卡牌数据管理、手牌管理、卡牌效果执行、卡牌池管理等。在战斗模块中被调用，通过事件系统通知卡牌UI更新。配置模块 (Config)：职责范围包括配置表的加载、缓存和访问接口封装。为其他所有模块提供数据访问服务，是系统数据驱动设计的核心支撑。UI模块 (UI)：职责范围包括各界面的显示逻辑管理。UI模块不直接调用业务模块，而是通过订阅事件被动更新，保证了UI与业务逻辑的解耦。

4.1.4 事件系统设计 事件系统是实现模块解耦的核心机制。系统基于GameFramework的Event模块，采用发布-订阅模式实现模块间通信。事件定义规范：每个事件对应一个继承自GameEventArgs的事件参数类，类名以EventArgs结尾。事件ID通过GetHashCode()自动生成，保证唯一性。事件参数类实现对象池接口，通过ReferencePool管理生命周期，避免GC分配。主要事件类型如表4.1所示：事件类名 触发时机 主要订阅方 携带数据

CombatStartEventArgs 战斗开始 CombatUI、BuffManager 触发类型、敌人信息 CombatEndEventArgs 战斗结束 GameProcedure、InventoryManager 胜负结果、奖励数据 ChessDeathEventArgs 棋子死亡 CombatManager、ChessManager 死亡棋子ID BuffAddedEventArgs Buff添加 BuffUI 棋子ID、BuffID BuffRemovedEventArgs Buff移除 BuffUI 棋子ID、BuffID BuffStackChangedEventArgs Buff层数变化 BuffUI 棋子ID、BuffID、新层数

ItemAddedEventArgs 物品添加 InventoryUI 物品ID、数量 ItemRemovedEventArgs 物品移除 InventoryUI 物品ID、数量 CardObtainedEventArgs 获得卡牌 CardUI 卡牌ID EnemyAlertChangedEventArgs 敌人警觉度变化 EnemyAlertUIManager 敌人ID、警觉度 CombatTriggerEventArgs 战斗触发 CombatProcedure 触发上下文 EnemyPurifiedEventArgs 敌人净化 CardManager 敌人ID、卡牌ID 表4.1 系统主要事件类型 事件的发布采用FireNow (立即触发) 和Fire (延迟到下一帧触发) 两种方式。对于需要立即响应的事件 (如伤害计算后的UI更新) 使用FireNow；对于可以延迟处理的事件 (如战斗结束后的奖励发放) 使用Fire，避免在事件处理中产生嵌套调用。

4.2 战斗系统设计

4.2.1 战斗系统整体架构 战斗系统采用自走棋机制，棋子在战斗中由 AI 自动控制完成移动、索敌与攻击等行为。其整体架构如图4.3所示，按职责分为流程控制层、实体管理层与战斗逻辑层三部分：流程控制层负责战斗生命周期与阶段切换，实体管理层负责棋子实体的创建、注册与数据同步，战斗逻辑层负责 AI 决策、技能执行、命中判定、伤害结算与战斗反馈。图 4.3 战斗系统架构 流程控制层由CombatManager负责，管理战斗的整体生命周期，包括战斗准备、战斗进行与战斗结算等阶段的统一编排，并协调战斗触发、脱战/逃脱与战场管理等子模块，确保战斗流程在不同场景下都能正确进入与退出。实体管理层由ChessManager负责，管理战场上棋子实体的生成与维护。由ChessFactory完成组件化创建与初始化；CombatEntityTracker负责实体追踪、阵营查询与胜负判定；BattleChessManager负责局内数据与全局状态的同步。每个棋子实体 (ChessEntity) 聚合AI、战斗控制器、属性与Buff等组件，以组件化方式组合出不同棋子能力。战斗逻辑层包含IChessAI的索敌与决策、ChessCombatController的动画与事件驱动执行、IChessSkill/IChessNormalAttack的技能与普攻逻辑、IHitDetector的命中检测，以及ChessAttribute的伤害计算与TakeDamage结算，并由BuffManager/BuffBase管理Buff生命周期。命中与受击后通过特效与伤害飘字模块提供反馈。

整个战斗过程由“决策—执行—命中—结算”链路驱动：AI在Tick中完成索敌与攻击决策；攻击决策由战斗控制器触发动画播放；攻击动画在执行帧通过AnimEvent_AttackExecute事件回调触发实际攻击；普攻/技能在执行时构建HitContext并调用命中检测器；命中检测器在命中回调中触发受击方ChessAttribute.TakeDamage完成伤害结算与事件派发，并驱动Buff、特效与飘字等反馈。4.2.2 棋子实体系统设计 棋子实体系统的类设计如图4.4所示，以ChessEntity为核心，通过组合模式整合多个功能组件，实现了棋子战斗单位的完整功能：图 4.4 棋子实体系统类图 棋子实体采用组件化合成设计：ChessEntity与ChessAttribute、ChessAnimator、ChessCombatController、BuffManager、ChessEXPComponent形成合成关系，各组件与棋子实体共享生命周期，棋子销毁时同步回收。其中ChessAttribute负责管理全部数值属性 (生命值、攻击力、护甲等) 及伤害计算逻辑；BuffManager负责棋子上所有Buff的生命周期管理；ChessCombatController负责接收动画事件回调并转发给对应技能，实现动画驱动与逻辑执行的解耦。

棋子的行为采用接口驱动的策略模式：AI决策 (IChessAI)、移动 (IChessMovement)、主动技能 (IChessSkill)、普攻 (IChessNormalAttack) 和被动技能 (IChessPassive) 均以接口形式定义，ChessEntity持有这些接口的引用。具体实现类由ChessFactory在初始化时根据配置表ID动态创建并注入，使不同棋子能够拥有差异化的AI行为与技能效果，同时对外保持统一的管理接口。运行时通过上下文对象ChessContext传递共享数据：ChessContext封装了技能和AI执行所需的全部引用 (棋子实体、属性组件、Buff管理器、阵营标识等)，在初始化阶段由ChessEntity构建后统一传递给各技能和AI实例，消除了组件间的直接依赖，有效降低了系统耦合度。ChessFactory作为静态工厂类，统一负责IChessAI、IChessSkill、IChessNormalAttack、IChessPassive四类接口实现的实例化。ChessEntity在初始化时调用ChessFactory，依据配置表中的技能ID和AI类型创建对应实例，将对象的创建逻辑与使用逻辑彻底分离，提升了系统的可维护性与扩展性。

高度疑似
AIGC (97.7%)

高度疑似
AIGC
(99.99%)

棋子属性系统设计如表4.2所示，属性分为基础属性和衍生属性两类：属性分类 属性名 类型 说明 基础属性 MaxHP int 最大生命值，从配置表读取 基础属性 AttackPower int 基础攻击力 基础属性 Defense int 物理防御力 基础属性 MagicResist int 魔法抗性 基础属性 AttackSpeed float 攻击速度（次/秒） 基础属性 MoveSpeed float 移动速度（米/秒） 基础属性 AttackRange float 攻击范围（米） 基础属性 CriticalRate float 暴击率（0-1） 基础属性 CriticalDamage float 暴击伤害倍率 衍生属性 CurrentHP int 当前生命值，运行时计算 衍生属性 FinalAttack int 最终攻击力（含Buff加成） 衍生属性 FinalDefense int 最终防御力（含Buff加成）

表4.2 棋子属性设计 棋子属性的修改通过ChessAttribute提供的接口进行，所有属性变化都会触发对应的事件通知，确保UI能够及时更新。Buff对属性的修改通过属性修改器（AttributeModifier）实现，支持固定值加成和百分比加成两种方式，修改器在Buff移除时自动撤销。棋子的初始化流程：ChessManager根据配置表ID创建ChessEntity实例→从SummonChessTable读取棋子配置→初始化ChessAttribute→通过SkillFactory创建技能实例→根据配置创建对应类型的ChessAI→完成初始化。 4.2.3 Buff系统设计 Buff系统是战斗系统中最复杂的子系统之一，负责管理战斗中各种增益（Buff）和减益（Debuff）效果。Buff系统的类图如图4.5所示。图 4.5 Buff系统类图 IBuff接口定义了Buff的完整生命周期与叠层行为：Init负责注入上下文与配置数据，OnEnter/OnUpdate/OnExit对应生效、每帧更新与移除三个阶段，OnStack/ReduceStacks处理叠层的增加与消耗，IsFinished则向管理器标识该Buff是否应被移除。该接口确保所有Buff类型遵循统一的运行规范，便于管理器进行统一调度。

BuffBase作为抽象基类实现了IBuff，内置持续时间倒计时与周期触发（Interval）的通用逻辑，并提供默认的叠层处理与持续时间刷新机制。子类可通过覆写OnTick与OnStackCountChanged，分别实现“周期性效果触发”和“层数变化时的状态反馈”等特化行为，在复用公共逻辑的同时保持各自的效果独立性。具体Buff通过继承BuffBase并覆写生命周期方法实现多样化效果。StatModBuff读取BuffTable中的StatMods字段（JSON格式），在生效与移除时对ChessAttribute执行属性修改的应用与撤销；BurnBuff、PoisonBuff、BleedBuff基于BuffTable.Interval周期触发伤害或持续性效果；StunBuff、SilenceBuff、TauntBuff通过BuffTable.CustomData（JSON格式）表达控制类或特殊状态逻辑；ShieldRegenBuff、MaxHpBuff、ReflectDamageBuff则分别实现护盾恢复、最大生命值提升与反伤等特殊机制。

BuffManager挂载于实体对象上，负责Buff的完整运行时管理，包括添加、移除、查询及每帧OnUpdate调度。其内部维护“激活列表”与“休眠列表”两套数据结构，支持带ActivationCondition的条件型Buff在满足触发条件时自动激活、不满足时进入休眠，从而实现条件触发与暂停机制。Buff实例的创建统一由BuffFactory完成，所需参数由BuffTable提供，最终形成完整的数据驱动Buff框架。Buff的配置通过BuffTable管理，主要字段如表4.3所示：字段名 类型 说明 Id int Buff唯一ID Name string Buff名称（用于UI显示） Description string Buff描述文本 BuffType int Buff类型（0=属性修改，1=持续伤害，2=控制，3=护盾） Duration int 持续回合数（-1表示永久，直到手动移除） MaxStack int 最大叠加层数（1表示不可叠加）

EffectParams string 效果参数，JSON格式，根据BuffType解析 IconId int 图标资源ID，用于BuffUI显示 IsInitiativeBuff bool 是否为先手Buff（战斗触发时使用） IsSneakDebuff bool 是否为偷袭Debuff（偷袭时可选择施加） 表4.3 BuffTable配置字段 4.2.4 技能系统设计 技能系统采用“策略模式+工厂模式+组件化聚合”的设计[10][28]：AI通过技能释放策略（ISkillReleaseStrategy）决定释放技能的时机与优先级；技能/普攻/被动的具体执行流程由各自实现类负责，从而将技能逻辑从战斗控制器中解耦出来。ChessFactory统一负责各类型对象的注册与创建，保证扩展新棋子技能时只需新增实现类并注册对应ID即可。技能系统的类图如图4.6所示：图 4.6 技能系统类图

技能/普攻执行由“AI决策→动画事件驱动执行→命中检测→结算反馈”构成：AI只负责决策与触发（Attack/Skill），战斗控制器负责播放动画并在执行帧回调时将执行权交给具体普攻/技能类；普攻/技能类构建HitContext并调用命中检测器完成命中判定；命中检测基类统一负责伤害结算、命中时Buff与命中回调，从而保证不同命中方式（投射物/近战/AOE等）复用同一结算逻辑。技能执行流程如图4.7所示：图 4.7 技能执行时序图 命中检测系统设计图如图4.8所示：图 4.8 命中检测系统详细设计 命中检测系统以IHitDetector作为统一抽象，使用HitDetectorFactory按攻击类型（AttackHitType）创建并复用具体检测器；各检测器只关心“如何命中”，而命中后的通用结算（受击特效、伤害结算、命中时Buff与回调）统一收敛在HitDetectorBase.ApplyDamage中，从而保证不同武器形态与技能类型在结算行为上保持一致，并减少重复代码。

命中检测系统支持五种检测类型，通过HitDetectorFactory工厂类创建：类型枚举 检测器类 适用场景 实现方式 Instant InstantHitDetector 瞬发单体攻击 直接对锁定目标应用伤害 Melee MeleeHitDetector 近战挥砍 武器Collider触发碰撞检测 Projectile ProjectileHitDetector 弓箭、投射物 生成投射物GameObject，碰撞时触发 AOE AOEHitDetector 范围技能 Physics.OverlapSphereNonAlloc检测范围内目标 Raycast RaycastHitDetector 激光、穿透攻击 Physics.Raycast射线检测 表4.4 命中检测类型 HitContext是命中检测的数据载体，包含攻击者、目标、伤害值、是否暴击、技能配置、命中回调等信息。命中检测器在检测到命中后，调用HitDetectorBase.ApplyDamage()方法，按照固定顺序执行：播放命中特效→应用伤害→执行配置表Buff（BuffIds字段）→触发OnHitCallback回调。 4.2.5 AI决策系统设计

棋子AI系统负责棋子的自主决策，包括目标选择、移动决策和攻击时机判断。AI系统采用简单高效的决策逻辑，在保证游戏体验的同时控制计算开销。Jagdale等（2021）系统总结了有限状态机在游戏AI开发中的应用方法[10]，本系统借鉴了其中的设计模式。AI类型通过配置表指定，支持三种基础AI类型：- RangedAI（远程AI）：优先保持与目标的距离，在攻击范围边缘进行攻击，避免进入近战范围。- MeleeAI（近战AI）：主动追击目标，尽快进入攻击范围，优先攻击距离最近的敌人。- SupportAI（辅助AI）：优先为血量最低的友方棋子提供治疗或增益，其次攻击最近的敌人。AI决策流程如图4.9所示，每帧执行以下步骤：

图 4.9 AI决策流程 第一步，目标搜索。从敌方棋子列表中筛选存活棋子，按照优先级排序（距离最近 > 血量最低 > 威胁度最高），选取最优目标。若无可用目标则进入待机状态。第二步，冷却检查。检查攻击冷却计时器，若仍在冷却中则跳过攻击，继续移动向目标靠近。第三步，距离判断。计算与目标的距离，若超出攻击范围则向目标移动；若在攻击范围内则执行攻击。第四步，攻击执行。缓存目标和预计算伤害，触发攻击动画，重置冷却计时器。攻击缓存机制是AI系统的重要

中度疑似
AIGC
(83.53%)

高度疑似
AIGC (100%)

高度疑似
AIGC
(99.91%)

高度疑似
AIGC (100%)

高度疑似
AIGC (97.7%)

高度疑似
AIGC
(99.33%)

要设计：AI在触发攻击时将目标和伤害值缓存到成员变量中，动画事件触发时从缓存中读取，避免了动画播放期间目标死亡或移动导致的逻辑错误。

4.2.6 伤害计算系统设计 伤害计算系统负责将攻击行为转化为最终扣血数值，支持物理伤害、魔法伤害和真实伤害三种类型，并依次处理暴击判定、Buff加成、防御减伤等修正环节。完整的计算流程如图4.10所示：图 4.10 伤害计算流程图 系统首先根据技能配置判定伤害类型，进入对应的分支处理。物理伤害与魔法伤害的计算流程相同，区别仅在于减伤所依据的防御属性不同——前者参考防御力，后者参考魔法抗性。真实伤害则跳过防御减伤环节，直接以基础伤害值作为最终结果。基础伤害计算完成后，系统生成随机数与攻击者的暴击率进行比较，命中暴击时将伤害值乘以暴击倍率。随后检查攻击者是否携带增伤类Buff，若存在则叠加对应的增伤倍数。两项修正按顺序独立应用，互不干扰。

对于物理伤害和魔法伤害，系统按公式 $\text{减伤率} = \text{防御力} / (\text{防御力} + 100)$ 计算防御减伤率， $\text{最终伤害} = \text{修正后伤害} \times (1 - \text{减伤率})$ 。该非线性公式使防御力的边际收益随数值增长而递减，无论防御力堆叠至多高，减伤率始终无法达到100%，从根本上避免了完全免疫的极端情况。真实伤害在完成暴击判定与Buff修正后，直接将修正后的数值作为最终伤害输出，不经过防御减伤计算。这一机制使特定穿透型技能能够无视目标的防御体系，在对抗高防御目标时保持稳定的输出效果。

4.2.7 战斗触发与脱战机制设计 战斗触发系统负责探索场景中战斗的发起，支持三种触发方式，通过CombatTriggerManager统一处理。偷袭（SneakAttack）触发条件：玩家与敌人距离小于3米；玩家位于敌人背后（玩家方向与敌人朝向夹角大于120度）；敌人警觉度低于0.3；玩家面向敌人（夹角小于45度）。偷袭成功后，玩家可从预设的Debuff池中选择一个施加给敌人，获得战斗优势。

遭遇战（Encounter）触发条件：玩家与敌人距离小于5米；敌人警觉度低于0.5；不满足偷袭条件。遭遇战触发后，玩家随机获得一个先手Buff。敌方先手（EnemyInitiated）触发条件：敌人追击玩家并进入战斗距离（CombatDistance）。敌方先手时，敌人随机获得一个先手Buff。CombatTriggerContext是战斗触发的数据载体，包含触发类型、触发敌人、可选Debuff列表、先手Buff ID、是否玩家先手等信息，在战斗准备阶段被CombatPreparationState读取并应用效果。脱战系统（CombatEscapeSystem）允许玩家在战斗中尝试逃跑。脱战成功率计算公式： $\text{成功率} = \text{基础成功率} + \text{当前回合数} \times \text{每回合加成}$ ， $\text{成功率上限} = \text{MaxSuccessRate}$ （配置表设定）脱战成功：增加污染值（CorruptionCost），返回探索场景，不进入冷却。脱战失败：损失当前生命值的HealthLossPenalty%，进入冷却（CooldownTurns回合内无法再次尝试）。

脱战参数通过EscapeRuleTable配置，支持针对不同敌人类型（普通/精英/Boss）设置不同的脱战难度。 4.3 物品与背包系统设计 4.3.1 物品系统整体架构 物品系统采用分层的E-R模型（如图4.6所示）。Player与Inventory形成一对一关系，一名玩家对应唯一的背包实例；Inventory包含多个InventorySlot，每个槽位通过外键引用具体的物品实例，并记录堆叠数量，支持同类物品的合并存储。图 4.11 物品系统E-R图 所有物品类型继承自BaseItem基类，在统一的基础字段之上扩展为Equipment（装备）和Treasure（宝箱）两大子类。Equipment新增装备槽位、属性修正等字段，为装备穿戴与属性加成提供数据支撑；Treasure新增稀有度、开启消耗和掉落表等字段，驱动宝箱开启与奖励生成机制。这种继承结构使各物品类型在共享基础逻辑的同时，能够独立扩展各自的专属字段。

Equipment与EquipmentAffix形成聚集关系，每件装备可拥有多个词条实例。词条记录属性类型、强度等级和具体数值，在装备生成与升级时参与计算。词条作为装备的内在组成部分，其生命周期与所属装备保持一致，不独立于装备单独存在。这一E-R模型通过规范化的表结构消除了数据冗余，各实体职责清晰、边界明确。继承关系保证了物品基础逻辑的复用，聚集关系则支撑了词条系统的灵活扩展。当需要新增物品类型时，只需继承BaseItem并补充专属字段，无需改动现有结构，具备良好的可扩展性。 4.3.2 物品基础类设计 ItemBase是所有物品的抽象基类，定义了物品的通用属性和行为接口：通用属性：ItemId（物品ID）、Name（名称）、ItemType（类型枚举）、Quality（品质，1-5对应普通到传说）、Description（描述文本）、IconId（图标资源ID）、DetailIconId（详情图标ID）、SellPrice（售价）。

通用行为：CanUse()（是否可使用）、CanEquip()（是否可装备）、CanStack()（是否可堆叠）、GetMaxStackCount()（最大堆叠数）。各子类在ItemBase基础上扩展特有属性：ConsumableItem（消耗品）：增加UseEffectId（使用效果ID）字段，使用时通过ItemEffectExecutor执行对应效果。QuestItem（任务道具）：增加DetailImageId（详情图片ID）字段，用于在物品详情界面显示大图。TreasureItem（宝物）：增加SpecialEffectId（固定特效ID）、AffixList（词条列表，运行时生成）、SynergyIds（羁绊ID列表）字段。宝物获得时从词条池随机生成1-3个词条，词条固定后不再变化。EquipmentItem（装备）：增加EquipType（装备类型，如武器/防具/饰品）、BaseAttributes（基础属性加成，JSON格式）、SpecialEffectId（特殊效果ID）字段。

4.3.3 背包容器系统设计 背包管理系统的类设计如图4.12所示，采用三层数据结构：InventoryManager→InventorySlot→ItemStack，各层职责清晰，通过合成关系保持生命周期一致。图 4.12 背包管理系统类图 InventoryManager作为单例管理器，持有固定数量的InventorySlot列表（默认100格），对外提供AddItem、RemoveItem、MoveItem、SortInventory等操作接口。所有操作完成后通过OnSlotChanged事件统一通知UI层更新，实现了数据与视图的解耦。InventorySlot是格子的数据载体，内部封装ItemStack对象而非直接存储ItemId和Count，这一设计将“物品是什么”与“有多少个”的逻辑集中在ItemStack中处理，格子本身只负责索引定位和空格判断。AddItem操作的核心逻辑分两步：首先调用TryStackItem尝试向已有同类格子追加数量；若无法堆叠或堆叠后仍有剩余，则调用FindEmptySlot寻找空格子创建新的ItemStack；背包已满时触发事件通知UI显示提示。MoveItem操作支持两格互换与同类物品合并堆叠，由InventorySlot.CanMergeWith判断是否可合并。SortInventory按物品类型和品质双关键字排序，空格子统一移至末尾。

4.3.4 物品堆叠机制设计 物品堆叠机制需要处理以下边界情况：单次添加超过最大堆叠数：将物品分配到多个格子，直到全部放入或背包满。添加到已有堆叠时超出上限：先填满现有堆叠，剩余部分创建新堆叠。移除数量超过当前堆叠数：从当前格子移除后，若数量不足则继续从其他同ID格子移除。 4.3.5 拖拽交互系统设计 拖拽交互系统基于Unity的EventSystem实现，支持以下交互场景：背包内拖拽：物品在背包格子间移动或交换位置。背包到装备槽：将装备或宝物拖拽到对应装备槽，触发装备操作。装备槽到背包：将已装备的物品拖回背包，触发卸下操作。拖拽到丢弃区域：弹出确认对话框，确认后从背包移除物品。

高度疑似
AIGC
(99.99%)

高度疑似
AIGC
(99.92%)

高度疑似
AIGC
(95.78%)

高度疑似
AIGC
(99.99%)

高度疑似
AIGC (100%)

高度疑似
AIGC (99.9%)

高度疑似
AIGC (100%)

高度疑似
AIGC
(99.48%)

拖拽系统的状态机设计：Idle（待机）→ Dragging（拖拽中，显示拖拽图标）→ Dropping（放置，检测目标类型并执行操作）→ Idle。拖拽图标（DragIcon）使用对象池管理，避免频繁创建销毁。拖拽过程中，原格子显示半透明状态，目标格子高亮显示，提供清晰的视觉反馈。

4.3.6 词条与羁绊系统设计 词条系统（Affix System）为宝物类物品提供随机属性加成。词条生成流程：宝物获得时，从AffixTable中筛选该宝物的词条池（AffixPoolIds字段指定）；使用加权随机算法（Weight字段控制权重）抽取1-3个词条；生成词条实例，确定具体数值（在ValueRange范围内随机）；词条数据存储在TreasureItem实例中，持久化到存档。词条效果通过AffixEffect类实现，支持属性加成（AttributeType指定属性，ValueType指定固定值或百分比）和特殊效果（通过EffectId关联SpecialEffectTable）两种类型。

高度疑似
AIGC（100%）

羁绊系统（Synergy System）在特定宝物组合装备时激活额外效果。羁绊检测由SynergyDetector负责，在装备/卸下宝物时触发检测：遍历所有已装备宝物的SynergyIds字段，统计每个羁绊ID出现的次数；若某羁绊ID出现次数达到SynergyTable中的RequireCount，则激活该羁绊效果；羁绊效果通过ItemEffectExecutor执行，效果类型为SynergyEffect。

4.4 策略卡系统设计 4.4.1 卡牌系统整体架构 卡牌系统的类设计（如图4.13所示）采用“卡牌+效果”的二层模型：卡牌是玩家交互的对象，封装费用、目标类型等属性；效果是卡牌的执行实体，通过多态机制实现伤害、Buff施加、治疗等差异化行为：图 4.13 卡牌系统类图 Card基类定义了卡牌的通用接口与公共属性，DamageCard、BuffCard、HealCard等子类在此基础上实现各自的特化逻辑。这种继承结构使卡牌类型的扩展只需新增子类，无需修改现有代码，具备良好的开放性。

高度疑似
AIGC（100%）

CardEffect作为抽象基类，定义了效果执行的统一接口。DamageEffect、BuffEffect等具体效果类继承CardEffect并各自实现OnExecute()方法，将“如何执行”的逻辑封装在子类内部。一张卡牌可以携带多个CardEffect实例，出牌时依次触发，从而组合出复合型效果。这种设计将效果的种类与卡牌本身解耦，新增效果类型时只需扩展CardEffect的子类，不影响卡牌层的逻辑。CardConfig存储卡牌的全部参数及其关联的效果列表，CardManager在运行时读取配置并动态生成对应的卡牌实例。数据与逻辑的分离使策划人员可以通过修改配置表调整卡牌行为，而无需改动代码，降低了内容迭代的成本。

4.4.2 卡牌数据管理设计 卡牌数据通过CardTable配置表管理，主要字段如表4.5所示： 字段名 类型 说明

高度疑似
AIGC
（99.03%）

Id int 卡牌唯一ID Name string 卡牌名称 Description string 卡牌效果描述（支持参数占位符） CardType int 卡牌类型（0=战斗强化，1=战场控制，2=资源获取） EffectType string 效果类型标识符，对应CardEffectExecutor中的工厂键 EffectParams string 效果参数，JSON格式，由具体效果类解析 IconId int 卡牌图标资源ID Rarity int 稀有度（1=普通，2=稀有，3=史诗，4=传说） UseCondition int 使用条件（0=无限制，1=仅战斗中，2=仅探索中） MaxCount int 手牌中最大持有数量

表4.5 CardTable配置字段

4.4.3 卡牌效果执行引擎设计 卡牌效果的执行由CardEffectExecutor统一调度，采用命令模式设计[28]。García-Sánchez等（2018）在《炉石传说》的自动化测试研究中验证了数据驱动卡牌效果执行的有效性[9]，本系统借鉴了该方法。

ICardEffect └─ Execute(DRCardTable config, CombatContext ctx) CardEffect接口定义了统一的执行契约：Init(CardData cardData)负责注入卡牌数据，Execute(Vector3targetPosition)负责在指定位置触发效果。卡牌执行流程如图4.14所示：图 4.14 卡牌执行流程图 当玩家将卡牌拖拽至战场区域释放时，系统首先检查灵力是否充足；满足消耗条件后扣除灵力，随即进入效果执行分支。CardEffectExecutor根据卡牌ID查询内部的效果类型映射表：对于少数具有复杂联动逻辑的特殊卡牌（如生命汲取、不屈意志），通过Activator.CreateInstance动态实例化对应的专用效果类；其余卡牌则走通用框架，由GetTargetSelector根据CardTargetType枚举选取目标选择器，由GetEffectAppliers根据配置表字段（DamageType、HealAmount、HitBuffIds等）组装效果应用器列表，最终构建`GenericCardEffect`实例执行。

高度疑似
AIGC（100%）

效果执行时，各效果类通过CardEffectHelper工具类访问底层战斗系统，该工具类提供DealDamage（伤害计算）、HealTarget（生命恢复）、ApplyBuff（Buff施加）、PlayEffect（特效播放）四个标准化静态接口，屏蔽了底层系统的调用细节。效果执行完成后，从CardManager中移除已使用的卡牌。目标选择通过CardTargetType枚举的8种类型覆盖全部战斗场景，单体和范围类型结合释放位置与配置表中的AreaRadius参数筛选最近目标。已实现的卡牌效果类型如表4.6所示： 卡牌ID 效果类名 名称 目标类型 效果说明 1001 HolyShieldCardEffect 神圣庇护 全体友方 为所有友方单位施加护盾Buff，吸收伤害 1002 FlameStormCardEffect 烈焰风暴 敌方全体 对敌方全体造成魔法伤害，命中时附加灼烧Buff 1003 TimeRewindCardEffect 时间回溯 单体友方 在释放位置范围内查找最近的友方单位，恢复其生命值

中度疑似
AIGC
（70.57%）

1004 WarCryCardEffect 战争号角 全体友方 提升全体友方攻击力和移动速度 1005 ShadowAssaultCardEffect 暗影突袭 单体敌方 对范围内最近的敌人造成高额物理伤害并施加眩晕 1006 LifeDrainCardEffect 生命汲取 敌方全体 对敌方全体造成魔法伤害，将总伤害的50%转化为治疗量，恢复血量最低的友方单位 1007 FrostNovaCardEffect 冰霜新星 范围内敌方 对释放位置范围内的所有敌方施加冻结Buff 1008 BerserkCardEffect 狂暴 自身 使释放位置最近的友方单位进入狂暴状态，大幅提升攻击力但降低防御 1009 GroupHealCardEffect 群体治疗 全体友方 恢复所有友方单位的生命值，治疗量从配置表读取 1010 ThunderStrikeCardEffect 雷霆一击 单体敌方 对范围内最近的敌人造成真实伤害，无视防御减免 1011 ChaosCurseCardEffect 混乱诅咒 敌方全体 对敌方全体施加混乱Buff，使其有概率攻击队友

1012 ResurrectionCardEffect 不屈意志 单体友方 复活范围内最近的已阵亡友方单位，恢复其50%最大生命值

表4.6 卡牌效果类型 卡牌效果与Buff系统通过两种方式集成：InstantBufs（释放时立即施加，如护盾、攻击力提升）和HitBufs（命中时触发，如灼烧、眩晕、冻结）。效果参数通过CardTable配置表的ParamsConfig字段以JSON格式存储，各效果类通过CardData.GetParam()方法读取，支持治疗量（healAmount）、回复比例（healRatio）、复活血量比例（reviveHpRatio）等灵活参数配置。新增卡牌效果只需实现ICardEffect接口并在CardEffectExecutor的映射表中注册，无需修改现有代码，符合开闭原则。

4.4.4 卡牌池管理设计 卡牌池采用分层设计，不同来源对应不同的卡牌池：净化普通敌人：从CommonCardPool抽取，包含普通和稀有卡牌。

高度疑似
AIGC
（99.85%）

净化精英敌人：从EliteCardPool抽取，包含稀有和史诗卡牌。净化Boss敌人：从BossCardPool抽取，包含史诗和传说卡牌。卡牌池的权重配置存储在CardPoolTable中，通过加权随机算法实现概率控制。为避免玩家重复获得相同卡牌，系统维护一个已获得卡牌的计数器，当某张卡牌数量达到MaxCount时，从当次抽取的候选列表中移除该卡牌。

4.5 探索系统设计 4.5.1 探索系统整体架构 探索系统核心类设计（如图4.15所示）围绕ExploreManager作为中心，协调玩家、敌人、掉落物品等游戏元素，实现了完整的探索功能。图 4.15 探索系统类图 Enemy实体集成AIDecisionSystem提供智能

行为。EnemyTriggerManager独立负责敌人的触发检测。通过配置驱动的方式，ExploreConfig定义场景的敌人生成和掉落规则，支持灵活的场景编辑。

4.5.2 敌人AI状态机设计

敌人AI的行为采用有限状态机（FSM）模式设计[10]（如图4.16所示），通过明确的状态定义与量化的转移条件，实现了结构清晰、行为可控的敌人AI逻辑。系统共定义空闲、巡逻、警戒、追击、战斗、死亡6个主要状态。图4.16 敌人AI状态机 巡逻状态下，敌人在其中定期扫描周围。当玩家进入视野后，敌人转移到警戒状态准备追击。若玩家距离足够近（<5m），敌人进入追击状态持续移动靠近。当距离足够近（<2m）时，进入战斗状态开始攻击。

追击状态设有两个退出条件：距离限制（>10m自动退出）和时间限制（30s超时自动退出）。这种设计防止了敌人的无限追击，同时保留了玩家的逃脱机会，提升了游戏的可玩性。所有转移条件都是基于可量化的参数（距离、时间、血量），这使得AI的行为可预测、可调试，也便于策划人员通过调整参数来平衡游戏难度。状态转换条件汇总如表4.7所示：当前状态 目标状态 转换条件 Idle Patrol 休息时间结束 Idle Alert 玩家进入AlertDistance Patrol Idle 到达巡逻点且随机触发 Patrol Alert 玩家进入AlertDistance Alert Chase 警戒时间结束且玩家仍在范围 Alert Patrol 警戒时间结束且玩家离开范围 Chase Combat 进入CombatDistance

Chase Patrol 玩家逃出ChaseDistance Combat Patrol 战斗结束（胜利/脱战）

表4.7 敌人AI状态转换条件

4.5.3 视野检测系统设计

视野检测系统（VisionConeDetector）采用双层检测机制，模拟敌人的听觉感知和视觉感知：视野检测系统设计如图4.17所示：图4.17 视野检测系统设计图 圆形感知范围以敌人中心，半径为VisionCircleRadius（默认8米），模拟听觉或环境感知。玩家进入此范围后，警觉度以AlertIncreaseRate（默认0.5/秒）的速率持续增长。扇形视觉范围以敌人朝向为轴，总张角VisionConeAngle（默认60度），距离VisionConeDistance（默认12米），模拟视觉感知。玩家进入扇形范围后，警觉度以AlertIncreaseRate×1.5的速率增长，体现视觉感知比听觉更为敏锐的设计意图。角度判断采用半角比较，即玩家与敌人朝向的夹角不超过30度时判定为在扇形内。

警觉度是[0,1]范围内的浮点数，每帧更新。玩家离开所有检测范围后，警觉度以AlertDecreaseRate（默认0.2/秒）的速率衰减。当警觉度达到1.0时，通知EnemyAI切换至Alert（警戒）状态。系统还内置了三种检测屏蔽机制：玩家战后隐身、玩家正在战斗中、全局过渡期屏蔽，触发任一条件时跳过检测并强制衰减警觉度，避免玩家从战斗返回后立即被再次触发追击。为优化性能，视野检测不在每帧执行，而是每0.15秒更新一次。物理检测使用Physics.OverlapSphereNonAlloc，复用缓存数组避免GC分配。

4.5.4 净化与奖励系统设计

净化系统在战斗胜利后触发，允许玩家对特定类型的敌人（Boss或Special类型）进行净化，获得策略卡牌奖励。净化流程：战斗胜利 → CombatManager发布CombatEndEventArgs事件 → EnemyEntityManager检查被击败敌人是否可净化（EnemyType为Boss或Special且PurificationCardId > 0） → 若可净化，显示净化UI（PurificationUI） → 玩家确认净化 → CardManager.AddCard(purificationCardId) → 发布CardObtainedEventArgs事件 → 净化UI关闭。

净化UI展示净化动画效果和即将获得的卡牌预览，增强玩家的获得感。净化操作是可选的，玩家也可以选择跳过净化直接返回探索。

4.6 数据表与配置系统设计

4.6.1 配置表 workflow 设计

配置表系统基于GameFramework的DataTable模块，采用Excel → TXT → .bytes的三阶段 workflow，实现了策划数据与程序代码的完全分离。 workflow 各阶段说明：第一阶段（Excel编辑）：策划人员在AAAGameData/DataTables/目录下的Excel文件中编辑游戏数据。Excel文件格式规范：第一行为字段注释（中文说明），第二行为字段名（英文，用于生成C#属性名），第三行为字段类型（int/float/string/bool等），第四行起为数据内容。第二阶段（代码生成）：运行DataTableGenerator工具（Tools/GameData/Generate DataTables），自动将Excel文件转换为TXT格式的数据文件和对应的C#数据类。生成的C#类存放在Assets/AAAGame/Scripts/DataTable/目录，命名规范为DR[TableName]（如DRBuffTable）。

第三阶段（二进制打包）：DataTableGenerator同时生成.bytes格式的C#二进制数据文件，存放在Assets/AAAGame/DataTable/目录。运行时加载.bytes文件而非TXT文件，提高加载速度。

配置表 workflow 中Excel编辑阶段的文件结构

如图4.18所示，以BuffTable为例，第一行为中文注释，第二行为英文字段名，第三行为字段类型，从第四行起为数据内容，每列宽度自适应以保证策划可读性。图4.18 BuffTable配置表Excel编辑示例

4.6.2 核心配置表设计

配置表驱动设计已成为现代游戏开发的标准实践[11][12]。系统涉及的主要配置表及其用途如表4.8所示：配置表名 主要字段 用途 SummonChessTable Id、Name、PrefabId、AttackHitType、各属性 棋子基础配置 SummonChessSkillTable Id、SkillType、DamageMultiplier、BuffIds、Cooldown 棋子技能配置

BuffTable Id、BuffType、Duration、MaxStack、EffectParams Buff效果配置 EnemyTable Id、Name、各战斗属性 敌人战斗配置 EnemyEntityTable Id、EnemyType、AI参数、视野参数、净化卡牌ID 敌人探索配置 ItemTable Id、Type、Quality、CanStack、UseEffectId 物品基础配置 SpecialEffectTable Id、EffectType、EffectParams 物品/技能效果配置 AffixTable Id、AttributeType、ValueType、ValueRange、Weight 词条配置 SynergyTable Id、RequireCount、RequireIds、EffectId 羁绊配置 CardTable Id、CardType、EffectType、EffectParams、Rarity 卡牌配置 CombatRuleTable Id、EnemyType、脱战参数 战斗规则配置 UITable Id、UIName、AssetPath、UIGroup UI界面配置

表4.8 核心配置表清单

4.6.3 配置表访问接口设计

为提高配置表访问的便利性和性能，系统设计了配置表扩展访问接口：缓存机制：对于频繁访问的配置数据（如BuffTable、ItemTable），实现静态缓存字典，首次访问时从DataTable加载并缓存，后续直接从缓存读取，避免重复查询。扩展方法：为常用配置表提供扩展方法，如BuffTableExtension.GetCached(id)、ItemTableExtension.GetCached(id)，简化调用代码。类型安全：所有配置表访问通过泛型接口GF.DataTable.GetDataTable<T>()进行，编译时检查类型安全，避免运行时类型错误。

4.7 UI系统设计

4.7.1 UI系统整体架构

UI系统基于GameFramework的UI模块构建[24]，采用表单（UIForm）模式管理所有界面。游戏UI设计需要平衡信息展示和用户交互流畅性[14][15][36]。Dickinson（2017）在Unity性能优化中探讨了UI元素对游戏性能的影响[14]，Llanos和Jørgensen（2011）探讨了玩家对集成式UI的偏好[15]。在这个基础上，我们把UI系统架构分为界面管理层、界面逻辑层和组件层三个层次进行实现。界面管理层由GF.UI组件负责，管理所有UI界面的打开、关闭、层级排序等。界面信息通过UITable配置表注册，包括界面名称、预制体路径、UI组（UIGroup）、排序层级等。

界面逻辑层包含各具体UI界面类，均继承自UIFormBase基类。UIFormBase提供标准的生命周期方法（OnInit、

高度疑似
AIGC
(99.95%)

高度疑似
AIGC (100%)

中度疑似
AIGC
(75.48%)

中度疑似
AIGC
(70.57%)

高度疑似

OnOpen、OnClose、OnUpdate)和通用功能(子UI管理、对象池管理、DOTween动画管理)。组件层包含可复用的UI组件,如物品格子(InventoryItemUI)、Buff图标(BuffIconItem)、棋子血条(ChessHPBar)等。这些组件通过UIFormBase提供的对象池接口管理,避免频繁创建销毁。4.7.2 UI组件引用管理设计 UI组件引用通过自动生成的Variables脚本管理,避免了手动使用Find或GetComponent查找组件带来的性能开销和维护问题。Variables脚本生成流程:在UIFormBase的Inspector中配置需要引用的组件→运行UI变量生成工具→自动生成[UIName].Variables.cs文件,存放在UIVariables/目录。Variables脚本中的组件引用在OnInit阶段通过序列化字段直接赋值,无需运行时查找,性能最优。命名规范:变量名以var前缀开头,如varBtnClose(关闭按钮)、varImgIcon(图标Image)、varTxtName(名称Text)。

4.7.3 背包UI设计 InventoryUI是背包界面的主要UI类,采用分页设计,支持大量物品的高效展示。分页机制:背包物品按页显示,每页固定数量(如20个格子)。PageLabelItem组件显示页码,支持点击切换页面。切换页面时,通过对象池回收当前页的物品格子,创建新页的物品格子,避免一次性创建所有格子带来的性能问题。物品格子

(InventoryItemUI):每个格子显示物品图标、数量、品质边框。点击格子显示物品详情面板(ItemDetailPanel),详情面板展示物品名称、描述、属性、词条等信息。右键菜单(ItemContextMenu):右键点击物品格子弹出上下文菜单,提供使用、装备、丢弃等操作选项,根据物品类型动态显示可用操作。

4.7.4 战斗UI设计 CombatUI是战斗场景的主要UI类,负责显示战斗状态信息。棋子血条(ChessHPBar):显示在棋子头顶,实时更新当前HP和最大HP的比例。血条颜色根据HP百分比变化(绿色→黄色→红色),提供直观的状态反馈。

Buff图标列表(BuffIconList):显示棋子身上的所有Buff图标,通过订阅BuffAddedEventArgs和BuffRemovedEventArgs事件动态更新。图标使用对象池管理,鼠标悬停显示Buff详情提示。伤害飘字

(DamageFloatingText):棋子受到伤害时,在棋子位置生成飘字特效,显示伤害数值。暴击伤害使用更大字号和特殊颜色区分。飘字通过GF.Entity.PopScreenText接口实现,使用对象池管理。4.7.5 战前准备UI设计 战前准备界面

(CombatPreparationUI)是战斗系统与物品系统的交汇点,承担棋子部署、装备预览、先手效果选择和准备倒计时四项功能。

棋子手牌区:棋子以扇形卡片方式排列,由ChessSlotContainer统一管理布局和进场动画。点击未出战棋子触发选中动画(卡片上移20px+脉冲缩放),同时侧边弹出DetailInfoUI展示棋子属性;点击已出战棋子仅显示详情,不允许取消部署。装备预览区:从背包过滤出装备类物品,显示在固定数量的装备槽(EquipSlotContainerImpl)中,背包数据变化时自动刷新。先手效果选择:偷袭触发时展示三选一Debuff面板,选项从SpecialEffectTable读取图标和描述,每个选项以80ms间隔错开弹出(缩放0→1,Ease.OutBack);先手Buff展示时开启5秒倒计时,超时自动选第一个,选中后播放脉冲确认动效,其他选项同步缩小淡出。4.7.6 战斗结算UI设计 战斗结算界面(EndCombatUI)根据战斗结果进入两条分支:

胜利分支:延迟一段时间(读取CombatRuleTable.VictoryTreasureDelaySeconds)后显示宝箱按钮。玩家点击宝箱后触发开箱动画(Animator.SetTrigger("Open")),动画关键帧触发SpawnAwards事件,随机生成2-6件奖励物品,每件以0.15秒间隔依次弹出(DOScale OutBack),全部展示完毕后显示"收取"按钮,点击将所有奖励批量写入背包。失败分支:自动计时(读取CombatRuleTable.DefeatAutoReturnDelaySeconds)后淡出界面并切换回探索流程,无需玩家操作。整个流程通过CancellationTokenSource与UniTask协调,界面关闭时立即取消所有挂起的异步操作,防止对象销毁后继续执行。4.7.7 主菜单UI设计 主菜单界面(StartMenuUI)提供新游戏、继续游戏、加载存档、设置和退出五个入口,并集成云存档入口。界面打开时播放多层次入场动画:背景淡入(0.6s)→中英文标题从上方滑入(OutBack,错开0.1s)→五个按钮从左侧交错滑入(每隔0.08s)→云存档按钮缩放弹出。动画进行到总时长50%时提前启用交互响应,兼顾视觉完整性与操作流畅性。OnClose时调用DOTween.Kill(this)并重置所有元素状态,保证对象池复用后初始状态正确。

4.7.8 UI动画设计 UI动画统一使用DOTween实现,通过DOTweenSequence组件在Inspector中配置动画参数,无需编写动画代码。界面打开动画:通常采用缩放+淡入组合(从0.8倍缩放+透明度0到1倍缩放+透明度1),持续0.2-0.3秒。界面关闭动画:与打开动画相反,关闭完成后调用GF.UI.CloseUIForm。注意事项:界面关闭时必须调用DOTween.Kill(target)或DOComplete()终止正在播放的动画,避免动画在界面关闭后继续执行导致的空引用错误。4.8 本章小结 本章对Clash of Gods游戏系统进行了详细的设计。在系统总体架构方面,采用四层架构设计,通过事件驱动架构实现模块解耦,设计了完整的事件系统规范。在战斗系统方面,详细设计了棋子实体系统、Buff系统、技能系统、AI决策系统、伤害计算系统和战斗触发机制,各子系统职责清晰、接口明确。在物品背包系统方面,设计了完整的物品继承体系、背包容器操作逻辑、拖拽交互状态机以及词条和羁绊系统。在策略卡系统方面,采用命令模式实现了可扩展的卡牌效果引擎。在探索系统方面,设计了五状态AI状态机、双层视野检测机制和净化奖励流程。在数据表系统方面,规范了配置表工作流和访问接口设计。在UI系统方面,设计了基于Variables脚本的组件管理方案和各主要界面的交互逻辑。本章的设计方案充分考虑了系统的可扩展性、可维护性和性能,为第5章的系统实现提供了完整的设计依据。5 系统实现 本章介绍Clash of Gods游戏系统各核心模块的具体实现过程。在第4章设计方案的指导下,结合Unity引擎和GameFramework框架的特性,对技术美术系统、战斗系统、物品系统、卡牌系统、探索系统、数据配置系统进行了详细实现,并针对PC端性能特点制定了系统性的优化策略。

5.1 技术美术系统实现 5.1.1 渲染管线配置 游戏采用Unity的通用渲染管线(Universal Render Pipeline,URP)作为渲染方案[21]。URP相比内置渲染管线具有更好的性能表现和更灵活的自定义能力[13],同时支持自定义着色器和后处理效果。URP渲染管线资产(URP Asset)的关键配置如下:关闭HDR(降低渲染开销);阴影距离设置为20米(超出范围不渲染阴影);主光源阴影分辨率设置为1024(平衡质量与性能);开启SRP Batchers(减少CPU渲染开销);开启GPU Instancing支持(相同棋子共享Draw Call)。渲染层(Rendering Layer)配置:Default层用于普通场景物体;Character层用于棋子和角色;UI层用于UI元素;Effect层用于粒子特效。通过层级分离,可以对不同类型的物体应用不同的渲染设置。

5.1.2 着色器系统实现 游戏实现了三套自定义URP着色器,满足不同的视觉需求[1][21]。章善举等(2025)研究了基于双缓冲区技术的图像渲染优化方案[1],本系统的渲染管线实现借鉴了该优化思路。角色描边着色器

AIGC (97.7%)

高度疑似
AIGC
(99.94%)

高度疑似
AIGC (100%)

高度疑似
AIGC
(99.85%)

高度疑似
AIGC (100%)

高度疑似
AIGC
(99.54%)

高度疑似
AIGC

(CharacterOutline.shader) 采用两Pass方案实现描边效果。第一Pass正常渲染角色；第二Pass将顶点沿法线方向外扩 OutlineWidth距离，仅渲染背面，颜色为OutlineColor。这种方案性能开销极小，且描边效果稳定。描边宽度和颜色通过材质参数控制，支持运行时动态调整，可用于高亮选中的棋子。角色描边效果对比如图5.1所示，左图为无描边状态，右图为选中状态（OutlineWidth=0.02，OutlineColor为高亮橙色）。图 5.1 角色描边着色器效果对比 溶解着色器 (Dissolve.shader) 用于角色死亡和技能特效。通过采样噪声贴图 (NoiseTexture) 获取溶解阈值，将阈值低于 DissolveProgress的像素丢弃 (clip)，在溶解边缘添加发光效果 (EdgeColor乘以EdgeWidth)。DissolveProgress从0到1的动画由DOTween驱动，实现平滑的溶解过渡效果。溶解效果如图5.2所示，展示了DissolveProgress从0到1过渡过程中的三个关键帧，边缘发光效果 (EdgeColor) 清晰可见。

图 5.2 溶解着色器过渡效果 卡通渲染着色器 (ToonLit.shader) 采用阶梯式光照模型，将漫反射光照分为三个区域：亮部 (NdotL大于0.5)、过渡区 (0.2到0.5之间)、暗部 (NdotL小于0.2)。每个区域使用固定的光照强度，产生卡通风格的色块效果。同时支持边缘光 (Rim Light) 效果，增强角色的立体感。卡通渲染效果如图5.3所示，从左到右分别是URP渲染管线-油画风格着色器渲染-油画卡通风格着色器渲染。图 5.3 卡通油画渲染着色器效果 5.1.3 粒子特效系统实现 粒子特效系统基于Unity的Particle System组件，通过CombatVFXManager统一管理特效的播放和回收。CombatVFXManager实现了特效对象池，预热常用特效（如命中特效、死亡特效）各10个实例，避免运行时频繁实例化带来的性能抖动。

技能特效：每个棋子的技能对应一套粒子特效预制体，特效预制体通过资源ID在SummonChessSkillTable配置表中引用，运行时通过ResourceExtension.LoadSpriteAsync异步加载，避免阻塞主线程。特效播放完成后自动回收对象池。命中特效：棋子受到攻击时播放命中特效，特效类型根据伤害类型（物理/魔法/暴击）有所不同，通过DamageFloatingTextTable配置表管理不同类型的特效参数。环境特效：探索场景中的环境特效（如火把、水流等）使用粒子系统实现，通过LOD（Level of Detail）技术在远距离降低粒子数量，优化性能。5.1.4 动画系统实现 棋子动画系统基于Unity的Animator组件，使用Animator Controller管理动画状态机。Base Layer包含Idle（待机）、Move（移动）、Attack（攻击）、Skill1（技能1）、Skill2（技能2）、Hit（受伤）、Death（死亡）七个状态。

移动动画使用混合树 (Blend Tree) 实现，根据Speed参数在Idle和Move动画之间平滑混合，避免移动开始/停止时的动画突变。动画事件实现是战斗系统的关键机制：在攻击动画的命中帧（通常是动画的40%到60%位置）添加Animation Event，事件函数名为AnimEvent_AttackHit。ChessCombatController实现该函数，在收到事件时调用技能类的执行方法，实现动画与逻辑的精确同步。这种设计确保了伤害判定与动画表现的一致性，避免了伤害提前或延迟触发的问题。如图5.4所示：图 5.4 动画事件帧配置示意图 5.2 战斗系统实现 5.2.1 战斗流程管理实现 CombatManager是战斗系统的核心管理器，通过有限状态机管理战斗的完整生命周期，包括准备阶段、战斗阶段和结算阶段。

战斗开始时，CombatManager接收CombatTriggerContext（战斗触发上下文），根据触发类型（偷袭/遭遇战/敌方先手）执行不同的先手效果处理。偷袭触发时，系统打开SneakDebuffSelectionUI界面，等待玩家从三个候选Debuff中选择一个施加给敌人；遭遇战和敌方先手触发时，系统随机选取一个先手Buff施加给对应方，并通过InitiativeBuffNotificationUI展示效果通知。先手效果处理完成后，进入战斗阶段。战斗阶段采用异步循环实现，每帧检查战斗是否结束（所有敌方棋子死亡或所有己方棋子死亡）。战斗结束后进入结算阶段，清除所有棋子的Buff效果，发布CombatEndEventArgs事件通知其他系统进行后续处理（如发放奖励、更新探索状态等）。整个战斗流程使用UniTask实现异步控制[22]，避免了协程的性能开销和难以调试的问题[16]。所有异步方法均以Async后缀命名，返回UniTask类型，符合项目的异步编程规范。Aversa和Dickinson（2019）详细分析了Unity协程的性能瓶颈，并推荐使用现代异步编程方案[16]。

5.2.2 棋子管理器实现 ChessManager负责战场上所有棋子的创建、更新和销毁。棋子通过GameFramework的Entity系统管理，利用实体系统内置的对象池减少实例化开销。棋子创建流程：根据配置表ID从SummonChessTable获取棋子配置→通过GF.Entity.ShowEntity创建实体→等待实体初始化完成→调用ChessEntity.Initialize初始化属性、技能和AI组件→将棋子添加到对应阵营列表。棋子更新在ChessManager的Update方法中统一执行，遍历所有存活棋子调用其Tick方法。这种集中更新的方式便于控制更新顺序，也便于在战斗暂停时统一停止所有棋子的更新。战斗结束时，ChessManager调用ClearAllBuffs清除所有棋子的Buff效果，然后通过GF.Entity.HideEntity隐藏棋子实体（回收对象池），为下次战斗做准备。

5.2.3 Buff管理器实现 BuffManager实现了Buff的完整生命周期管理，包括添加、叠加、每回合触发和移除。内部使用Dictionary<int, IBuff>存储活跃Buff，以BuffID为键，保证查询效率为O(1)。该设计模式在何柳青（2023）关于动作RPG战斗系统的研究中得到了验证[38]。Buff添加逻辑：首先从BuffTable获取配置；若该Buff已存在且层数未达上限，则调用AddStack增加层数并发布BuffStackChangedEventArgs事件；若不存在，则通过BuffFactory创建Buff实例，调用OnApply执行初始效果，并发布BuffAddedEventArgs事件通知BuffUI更新显示。Buff每回合触发：CombatManager在每回合结束时调用BuffManager.TickAllBuffs，遍历所有活跃Buff执行OnTick（如持续伤害Buff在此扣除HP），并收集已过期的Buff进行移除。Buff移除逻辑：调用IBuff.OnRemove撤销Buff效果（如移除属性修改器），从字典中删除，发布BuffRemovedEventArgs事件通知UI更新。

Buff应用支持两种方式以满足不同的技能设计需求：配置表方式通过SummonChessSkillTable的BuffIds字段配置，命中时由EffectExecutor.ApplyBuffsOnHit自动应用，适用于每次命中都触发的无条件Buff；回调方式通过技能的OnHitCallback手动调用，适用于需要条件判断的Buff，如后羿普攻在持有烈焰箭Buff时才附加灼烧效果。5.2.4 技能工厂与技能实现 技能工厂 (SkillFactory) 采用预注册字典方式，将技能ID映射到对应的技能类工厂函数，避免了反射带来的性能开销。新增技能只需在字典中注册对应的工厂函数，无需修改工厂类本身，符合开闭原则。每个棋子的技能类实现IChessSkill接口，封装完整的技能执行逻辑。以嫦娥普攻为例，该技能通过配置表方式自动应用寒霜Buff (BuffIds=2)，无需在代码中手动处理Buff逻辑，体现了配置表驱动设计的优势。

以后羿普攻为例，该技能通过回调方式实现条件性Buff应用：技能命中后检查后羿是否持有烈焰箭Buff (ID=4)，若持有则对目标附加灼烧Buff (ID=1)。这种设计将技能的条件逻辑完全封装在技能类中，不影响其他系统，体现了策略模式的

(89.32%)

高度疑似
AIGC (97.7%)

高度疑似
AIGC
(85.19%)

高度疑似
AIGC (97.7%)

高度疑似
AIGC
(99.91%)

高度疑似
AIGC (100%)

高度疑似
AIGC
(98.75%)

高度疑似
AIGC

优势。命中检测器（HitDetector）是技能执行的关键组件。HitDetectorFactory根据技能配置的AttackHitType字段创建对应的检测器实例。投射物检测器（ProjectileHitDetector）创建投射物GameObject并设置飞行轨迹，投射物碰撞到目标时触发命中逻辑；AOE检测器（AOEHitDetector）使用Physics.OverlapSphereNonAlloc检测范围内所有目标，对每个目标执行命中逻辑。5.2.5 AI决策算法实现 棋子AI的决策循环在ChessAI.Tick方法中实现[10]，每帧执行目标搜索和攻击决策。目标搜索采用线性遍历方式，从敌方棋子列表中找到距离最近的存活棋子作为攻击目标。

(99.85%)

攻击缓存机制是AI系统的重要设计：AI在触发攻击时将目标引用缓存到成员变量m_CachedTarget中，动画事件触发时从缓存中读取目标执行伤害逻辑。这种设计避免了动画播放期间目标死亡或移动导致的逻辑错误，确保伤害始终作用于正确的目标。移动实现使用NavMeshAgent组件[25]，通过设置destination属性驱动棋子向目标移动。NavMeshAgent的速度和加速度参数从配置表读取，支持不同棋子的移动特性差异化配置。AI类型通过配置表的AIType字段指定，ChessAI在初始化时根据AIType创建对应的目标选择策略：RangedAI优先保持与目标的距离，在攻击范围边缘进行攻击；MeleeAI主动追击目标，尽快进入攻击范围；SupportAI优先为血量最低的友方棋子提供治疗，其次攻击最近的敌人。

高度疑似
AIGC
(99.99%)

5.2.6 伤害计算公式实现 伤害计算在ChessAttribute.TakeDamage方法中实现，支持物理伤害、魔法伤害和真实伤害三种类型。物理伤害和魔法伤害受防御力/魔法抗性影响，采用非线性减伤公式：减伤率 = 防御力 / (防御力 + 100)。该公式的特点是防御力越高减伤率增长越慢，避免了防御力堆叠过高导致免疫伤害的问题。当防御力为100时减伤率为50%，防御力为200时减伤率约为67%，防御力为400时减伤率约为80%，符合游戏平衡性要求。真实伤害不受防御力影响，直接扣除目标HP，适用于穿透类技能和特殊效果。暴击判定在伤害计算前执行，若触发暴击则将基础伤害乘以暴击伤害倍率（CriticalDamage，从配置表读取）。暴击伤害以不同颜色和字号的飘字显示，给予玩家明确的视觉反馈。

高度疑似
AIGC (100%)

伤害计算完成后，通过GF.Entity.PopScreenText接口在棋子位置生成伤害飘字，并发布ChessHitEventArgs事件通知血条UI更新。当HP降至0时触发死亡流程，播放死亡动画并发布ChessDeathEventArgs事件。5.2.7 脱战系统实现 脱战系统（CombatEscapeSystem）在战斗UI中提供脱战按钮，玩家点击后触发脱战尝试。脱战成功率根据EscapeRuleTable配置的参数计算：基础成功率加上当前回合数乘以每回合加成，结果不超过最大成功率上限。脱战结果处理：成功时增加玩家的污染值（CorruptionCost），通过CombatEndEventArgs事件通知系统以脱战方式结束战斗，返回探索场景；失败时扣除玩家当前HP的HealthLossPenalty百分比，进入冷却状态（CooldownTurns回合内脱战按钮禁用），继续战斗。脱战冷却状态通过UI按钮的interactable属性控制，冷却期间显示剩余冷却回合数，提供清晰的状态反馈。

高度疑似
AIGC
(99.99%)

5.3 物品系统实现 5.3.1 物品工厂与物品类实现 物品系统采用工厂模式创建物品实例。ItemFactory根据物品ID的分段规则判断物品类型，创建对应的子类实例：ID在10000-19999范围创建ConsumableItem，20000-29999创建QuestItem，30000-39999创建TreasureItem，40000-49999创建EquipmentItem。TreasureItem（宝物）的创建需要额外的词条生成步骤。AffixGenerator从AffixTable中筛选该宝物的词条池，使用加权随机算法抽取1到3个词条，为每个词条在ValueRange范围内随机确定具体数值，生成AffixInstance列表存储在TreasureItem中。词条数据随物品一起持久化到存档，保证每次读取时词条不变。图 5.5 物品信息预览 图 5.6 物品详情信息显示 5.3.2 背包管理器实现 InventoryManager实现了背包的核心逻辑，内部使用List<InventorySlot>存储物品数据，列表大小固定为背包容量（MaxCapacity，从配置表读取）。AddItem操作的完整实现：首先从ItemTable获取物品配置，检查是否可堆叠；若可堆叠，遍历背包查找相同ID且数量未满的格子，找到则增加数量并发布ItemAddedEventArgs事件；若未找到可叠加格子，查找空格子（ItemId为0的格子），找到则创建新InventorySlot；若背包已满，发布InventoryFullEventArgs事件提示玩家，返回失败。RemoveItem操作：遍历背包查找包含目标物品的格子，减少数量；若数量降至0则清空格子（ItemId置为0，Count置为0）；发布ItemRemovedEventArgs事件通知UI更新。

高度疑似
AIGC
(97.96%)

SortItems操作：使用LINQ对背包格子进行排序，排序规则为：先按物品类型（消耗品→任务道具→宝物→装备），再按品质（传说→史诗→稀有→优秀→普通），空格子排在末尾。排序完成后发布InventorySortedEventArgs事件，触发UI全量刷新。背包显示效果如图5.7所示：图 5.7 背包显示效果 5.3.3 装备系统实现 EquipmentManager管理玩家的装备槽位，支持武器、防具、饰品、宝物四种装备类型，每种类型有固定数量的槽位。装备操作流程：检查物品是否可装备（CanEquip为true）且装备类型匹配目标槽位；若目标槽位已有装备，先执行卸下操作（将原装备放回背包）；将物品从背包移除并放入装备槽；调用EquipmentManager.ApplyEquipmentAttributes将装备属性加成应用到玩家属性；发布EquipmentChangedEventArgs事件通知UI更新。宝物装备时还需触发SynergyDetector.CheckSynergies检测羁绊条件，若满足羁绊激活条件则通过ItemEffectExecutor执行羁绊效果。

高度疑似
AIGC (100%)

5.3.4 拖拽交互系统实现 拖拽系统基于Unity的EventSystem，InventoryItemUI组件实现IBeginDragHandler、IDragHandler、IEndDragHandler三个接口。拖拽效果如图5.8所示：图 5.8 拖拽效果显示 拖拽开始（OnBeginDrag）：从对象池获取DragIcon实例，设置图标为当前物品图标，记录拖拽源格子索引；将原格子设置为半透明状态（alpha=0.5），提供视觉反馈。拖拽中（OnDrag）：更新DragIcon位置跟随鼠标/手指移动；检测当前鼠标位置下的UI元素，若为有效目标（背包格子/装备槽/丢弃区域）则高亮显示目标。拖拽结束（OnEndDrag）：通过EventSystem.RaycastAll检测放置目标；根据目标类型执行对应操作（背包格子间交换、装备到装备槽、丢弃）；恢复原格子透明度；将DragIcon回收对象池。丢弃操作需要弹出确认对话框（ConfirmDialog），玩家确认后才执行移除操作，防止误操作。

高度疑似
AIGC
(99.98%)

5.3.5 物品效果执行实现 ItemEffectExecutor根据SpecialEffectTable中的EffectType字段，将效果分发到对应的处理方法。效果参数通过JSON格式存储在EffectParams字段，由具体处理方法解析。已实现的效果类型包括：AddGold（获得金币，从EffectParams读取Min/Max范围随机获得）、RestoreHP（恢复生命值，支持固定值和百分比两种方式）、AddBuff（施加Buff，从EffectParams读取BuffId）、AddCard（获得卡牌，从EffectParams读取CardId）等。新增效果类型只需在EffectType枚举中添加新值，并在ItemEffectExecutor中添加对应的处理分支，无需修改其他代码，扩展成本极低。5.4 卡牌系统实现 5.4.1 卡牌管理器实现 CardManager管理玩家手牌的完整生命周期。手牌数据以List<int>（卡牌ID列表）形式存储在PlayerDataModel中，支持游戏存档和读取。获取卡牌：AddCard方法检查手牌中该卡牌的数量是否达到MaxCount上限，未达上限则添加到手牌列表，发布CardObtainedEventArgs事件通知CardUI更新显示。

高度疑似
AIGC
(98.75%)

使用卡牌：UseCard方法首先检查使用条件（UseCondition字段，如仅战斗中可用）；条件满足则调用

高度疑似

CardEffectExecutor.Execute执行卡牌效果；效果执行成功后从手牌列表中移除该卡牌，发布CardUsedEventArgs事件。5.4.2 卡牌效果执行器实现 CardEffectExecutor通过预注册的工厂字典实现效果的动态分发。工厂字典以EffectType字符串为键，以ICardEffect工厂函数为值，在系统初始化时注册所有已实现的效果类型。执行流程：从CardTable获取卡牌配置 → 从工厂字典查找对应的效果工厂 → 创建ICardEffect实例 → 调用Execute方法传入配置和战斗上下文 → 记录执行日志。以WarCryCardEffect（战吼）为例，该效果对所有友方棋子施加攻击力提升Buff，提升幅度从EffectParams的JSON数据中读取。效果执行时遍历ChessManager.PlayerChesses列表，对每个存活棋子调用BuffManager.AddBuff，实现批量Buff施加。

5.4.3 卡牌UI实现 CardUI展示玩家手牌，采用扇形布局排列卡牌，如图5.9所示：图 5.9 卡牌显示效果 选中卡牌时，通过DOTween动画将卡牌放大并上移，显示卡牌详情面板（名称、描述、效果预览）。点击使用按钮触发CardManager.UseCard，使用成功后通过DOTween动画将卡牌飞出屏幕，然后刷新手牌显示。5.5 探索系统实现 5.5.1 敌人AI状态机实现 EnemyEntityAI实现了基于有限状态机的敌人行为控制。状态机采用状态对象模式，每个状态封装为独立的类（EnemyIdleState、EnemyPatrolState、EnemyAlertState、EnemyChaseState），通过EnemyStateBase基类统一接口。状态切换通过ChangeState方法实现：调用当前状态的OnExit方法执行退出逻辑 → 创建新状态实例 → 调用新状态的OnEnter方法执行进入逻辑 → 更新当前状态引用。每次状态切换都通过DebugEx.Log记录日志，便于调试和问题定位。

EnemyEntityAI的Update方法每帧调用当前状态的OnUpdate，并调用UpdatePlayerDetection更新视野检测。视野检测使用计时器控制更新频率（每0.15秒一次），避免每帧执行物理检测带来的性能开销。5.5.2 视野检测系统实现 VisionConeDetector实现了双层视野检测机制。圆形感知检测使用Physics.OverlapSphereNonAlloc，将检测结果存储在预分配的缓存数组中，避免每次检测都分配新数组带来的GC压力。扇形视觉检测在圆形检测的基础上增加角度判断：计算玩家方向向量与敌人朝向向量的夹角，若夹角小于VisionConeAngle的一半则判定在扇形范围内。警觉度更新逻辑：玩家在圆形范围内时，警觉度以AlertIncreaseRate速率增长；玩家在扇形范围内时，警觉度以AlertIncreaseRate乘以1.5的速率增长；玩家离开所有检测范围时，警觉度以AlertDecreaseRate速率衰减。警觉度通过Mathf.Clamp01限制在0到1范围内。

当警觉度超过AlertThreshold时，通知EnemyEntityAI切换到Alert状态，并通过EnemyAlertUIManager显示警觉UI（感叹号图标）。EnemyAlertUIManager使用对象池管理警觉UI实例，按距离排序最多显示5个，避免UI过多影响性能。5.5.3 战斗触发器实现 CombatOpportunityDetector每0.1秒检测一次战斗触发条件，使用计时器控制检测频率。检测逻辑按优先级执行：先检测偷袭条件（距离、角度、警觉度三个条件同时满足），再检测遭遇战条件（距离和警觉度两个条件满足），若均不满足则隐藏交互提示UI。满足触发条件时，显示对应的交互提示UI（偷袭提示或遭遇战提示），并监听玩家的交互输入（通过PlayerInputManager.GetInteractDown）。玩家按下交互键后，CombatTriggerManager创建CombatTriggerContext，根据触发类型填充相应数据（可选Debuff列表、先手Buff ID等），然后通过GF.Event.FireNow发布CombatTriggerEventArgs事件，由CombatProcedure响应并切换到战斗场景。精英敌人的广播机制在EnemyChaseState中实现：追击开始时设置m_HasBroadcasted标志为false；当进入广播距离时，调用EnemyGroupManager.BroadcastPlayerPosition，通知范围内所有非战斗状态的敌人切换到Chase状态；广播完成后将m_HasBroadcasted置为true，确保每次追击只广播一次。

5.6 数据配置系统实现 5.6.1 配置表加载实现 配置表在PreloadProcedure阶段统一加载，通过AppConfigs ScriptableObject配置需要加载的表名列表。加载采用并行异步方式，所有配置表同时开始加载，等待全部完成后才进入下一流程，最大化利用IO并行性：private async UniTask LoadDataTablesAsync() { var tableNames = AppConfigs.Instance.DataTableNames; var tasks = tableNames.Select(name => GF.DataTable.LoadDataTableAsync(name, useBytes: true).AsUniTask()); await UniTask.WhenAll(tasks); DebugEx.Success("PreloadProcedure", \$"配置表加载完成，共 {tableNames.Length} 张"); } 加载完成后，各业务系统通过GF.DataTable.GetDataTable<T>()泛型接口访问配置数据，编译时检查类型安全。5.6.2 配置缓存机制实现 对于频繁访问的配置数据，实现了静态缓存机制，避免每次都从DataTable进行查询：

```
public static class BuffTableExtension { private static readonly Dictionary<int, DRBuffTable> s_Cache = new(); public static DRBuffTable GetCached(int id) { if (!s_Cache.TryGetValue(id, out var row)) { row = GF.DataTable.GetDataTable<DRBuffTable>().GetDataRow(id); if (row != null) s_Cache[id] = row; } return row; } // 场景切换时清空缓存（配置表重新加载后需要刷新） public static void ClearCache() => s_Cache.Clear(); } 缓存机制对BuffTable、ItemTable、CardTable等高频访问的配置表均有实现，在战斗场景中可显著减少DataTable查询次数。5.6.3 配置表通用访问接口实现 为屏蔽直接操作GF.DataTable的重复代码，系统在DataTableExtension中实现了一套通用的泛型访问接口，任意配置表均可通过同一套接口查询，无需为每张表编写重复逻辑：
```

```
public static T GetRowById<T>(int id) where T : class, IDataRow { var dataTable = GF.DataTable.GetDataTable<T>(); if (dataTable == null) { Log.Warning($"配置表 {typeof(T).Name} 未加载！"); return null; } return dataTable.GetDataRow(id); } public static List<T> GetRowsWhere<T>(System.Func<T, bool> predicate) where T : class, IDataRow { List<T> rows = new List<T>(); var dataTable = GF.DataTable.GetDataTable<T>(); if (dataTable == null) { Log.Warning($"配置表 {typeof(T).Name} 未加载！"); return rows; } var allRows = dataTable.GetAllDataRows(); foreach (var row in allRows) { if (predicate(row)) { rows.Add(row); } } return rows; }
```

配置表数据文件（.bytes格式）作为AssetBundle资源管理，支持按需加载。游戏启动时在PreloadProcedure阶段统一加载所有配置表，加载完成后各业务系统即可通过DataTableExtension访问配置数据。5.7 性能优化策略 5.7.1 CPU优化实现 对象池优化：频繁创建销毁的对象统一使用对象池管理。GameFramework的ObjectPool组件[24]为各系统提供对象池支持，包括特效对象池（CombatVFXManager）、UI Item对象池（UIFormBase.SpawnItem）、投射物对象池（ProjectilePool）等。对象池预热在场景加载时执行，避免战斗中首次创建对象带来的卡顿[13][14]。检测频率控制：非关键逻辑降低更新频率，减少CPU开销。视野检测每0.15秒执行一次，战斗机会检测每0.1秒执行一次，UI距离排序每

AIGC
(99.97%)

高度疑似
AIGC
(98.76%)

高度疑似
AIGC
(92.41%)

高度疑似
AIGC
(99.99%)

中度疑似
AIGC
(77.72%)

高度疑似
AIGC
(99.14%)

0.5秒执行一次，敌人巡逻点选取在到达目标点时才执行（非每帧）。

字符串优化：避免在Update等高频方法中进行字符串拼接。日志输出使用条件编译（#if UNITY_EDITOR）限制在编辑器模式下执行，发布版本不产生字符串分配。5.7.2 GPU优化实现 Draw Call合并：UI使用Canvas分组，相同材质的UI元素自动合批（Dynamic Batching）。角色模型开启GPU Instancing[26]，相同棋子的多个实例共享一次Draw Call，在多棋子战斗场景中效果显著[13]。LOD系统[27]：探索场景中的敌人实体根据与摄像机的距离切换LOD级别，远距离使用低精度模型（面数减少50%-70%）。Shader优化：着色器避免使用复杂的光照计算（如实时全局光照、屏幕空间反射等），使用预烘焙光照贴图（Lightmap）替代实时光照[21]，在保证视觉效果的同时大幅降低GPU开销[13]。

图 5.10 光照预烘焙示例 5.7.3 内存优化实现 资源异步加载：大型资源（模型、贴图、音频）使用异步加载，避免同步加载导致的主线程卡顿。不再使用的资源通过GF.Resource.UnloadAsset及时卸载，防止内存泄漏。纹理压缩：PC平台使用DXT5/BC7格式进行纹理压缩，在保证画质的同时减少内存占用约50%-70%。Sprite图集：UI图标使用Sprite Atlas打包，减少纹理数量和Draw Call。每个功能模块的图标打包为独立的图集（如BuffIconAtlas、ItemIconAtlas），按需加载，避免一次性加载所有图标。图 5.11 图集打包示例 5.8 UI系统实现 5.8.1 UI系统整体实现思路 所有UI继承自UIFormBase，状态感知型界面使用StateAwareUIForm子类实现。StateAwareUIForm在OnInit阶段统一订阅GameFramework事件，在OnClose阶段取消订阅，避免因漏取消订阅导致的内存泄漏。

UI组件引用通过自动生成的*.Variables.cs文件管理，所有变量以var前缀命名（如varHPSlider、varReadyBtn），在OnInit时由序列化字段直接赋值，无运行时查找开销。5.8.2 主菜单UI实现 主菜单界面如图5.12所示：图 5.12 主菜单界面 StartMenuUI的OnInit阶段通过CacheAnimationComponents遍历五个菜单按钮，为每个按钮缓存CanvasGroup、RectTransform和原始锚点坐标，存储到ButtonAnimData结构体数组中。入场动画通过单个DOTween.Sequence构建[23]，使用Sequence.Insert(time, tween)精确控制每个元素的起始时刻。UI元素的交互设计遵循Dickinson（2017）提出的UI优化原则[14]，并参考徐嘉良（2025）关于引导-反馈机制的研究[37]。具体的动效如下：背景淡入（0.6秒）→中英文标题从上方滑入（OutBack，错开0.1秒）→五个按钮从左侧交错滑入（每隔0.08秒）→云存档按钮缩放弹出。所有Tween共享同一Sequence实例，便于整体Kill，核心代码如下：

```
var sequence = DOTween.Sequence().SetTarget(this).SetUpdate(true);
sequence.Join(m_BgCanvasGroup.DOFade(1f, BG_FADE_DURATION)); sequence.Insert(TITLE_CN_DELAY,
m_TitleCnRect.DOAnchorPos(originalPos, TITLE_FADE_DURATION).SetEase(Ease.OutBack)); for (int i = 0; i <
m_ButtonAnimDatas.Length; i++) { float delay = BTN_GROUP_START_DELAY + i *
BTN_STAGGER_INTERVAL; sequence.Insert(delay, m_ButtonAnimDatas[i].canvasGroup.DOFade(1f,
BTN_FADE_DURATION)); } sequence.InsertCallback(sequence.Duration() * 0.5f, () => Interactable = true);
动画进行到总时长50%时提前启用交互（Interactable = true），兼顾视觉完整性与操作流畅性。OnClose时调用
DOTween.Kill(this)后立即执行ResetAnimationState，将所有元素恢复至alpha=1、原始锚点位置，确保对象池复用后
再次打开时初始状态正确。
```

5.8.3 战前准备UI实现 战前准备界面如图5.13所示：图 5.13 战前准备界面 棋子手牌区：InitializeChessPanelAsync逐一从对象池获取ChessItemUI，设置数据后调用ChessSlotContainer.AddCardAsync播放进场动画，每张棋子卡依次入场。拖拽开始时通过ChessPlacementManager.StartPlacementAsync启动Ghost预览系统，拖拽结束时调用ConfirmPlacementFromDrag直接确认放置，无需额外点击。先手效果选择：三个选项以手动计算的水平均匀间距排列，通过RectTransform.anchoredPosition直接设置位置（移除LayoutGroup以获得精确控制）。进场动画使用DOScale(1f, 0.25f).SetEase(Ease.OutBack).SetDelay(i * 0.08f)实现错落弹出效果。选中后的确认动效（PlayBuffSelectionConfirmAnimAsync）执行：选中项脉冲（1→1.15→1）→其他项同时缩小+淡出→等待300ms→清理GameObject→面板淡出。全程通过GetCancellationTokensOnDestroy绑定UI生命周期，UI关闭时自动中断。Buff选择界面如图5.14所示：图 5.14 战前Buff选择界面 先手Buff展示时开启5秒倒计时（AutoSelectBuffAfterTimeoutAsync），超时自动选取第一个选项，避免玩家因未及时操作而卡死流程。

准备倒计时：在OnUpdate中递减m_CountdownRemain，降至0时自动触发准备完成，读取CombatRuleTable.PreparationDurationSeconds配置，准备时间可由配置表灵活调整。5.8.4 战斗UI实现 战斗界面如图5.15所示：图 5.15 战斗界面 召唤师状态条：CombatUI通过SummonerRuntimeDataManager的OnHPChanged和OnMPChanged事件驱动更新，仅在数值变化时执行UI刷新，无需每帧Poll，CPU开销极低。MP回复在OnUpdate中每帧调用SummonerRuntimeDataManager.UpdateMPRegen(elapseSeconds)，保证回复曲线与帧率无关。手牌刷新：RefreshCardSlotsAsync异步执行，先调用CardSlotItemPool.ReturnCard批量回收旧卡，等待一帧（await UniTask.Yield）确保UI树更新，再从对象池获取新卡通过AddCardSilent静默添加，最后调用PlayAllCardAnimationsAsync统一播放进场动画（基于最终总卡数计算扇形位置），保证动画位置的正确性。刷新按钮：每场战斗最多刷新3次，消耗规则为：第1次消耗40%最大灵力，第2次消耗30%最大生命值，第3次消耗40%最大生命值，通过m_RefreshCount计数器控制，资源不足时阻止操作并输出提示。

召唤师技能按钮：RefreshSummonerSkillButtonsAsync从SummonerSkillManager读取技能列表，通过ResourceExtension.LoadSpriteAsync异步加载图标，按钮点击通过PlayerInputManager.TriggerSummonerSkill(slot)统一分发，符合项目全部输入走PlayerInputManager的规范。

5.8.5 战斗结算UI实现 战斗结算界面如图5.16所示：图 5.16 战斗结算界面 EndCombatUI.OnOpen从UIParams读取IsVictory参数决定展示分支。胜利分支：延迟CombatRuleTable.VictoryTreasureDelaySeconds秒后显示宝箱按钮。玩家点击后触发Animator.SetTrigger("Open")，动画关键帧事件SpawnAwards触发奖励生成

（GenerateItemAwardsAsync），每件奖励以0.15秒间隔依次弹出（DOScale OutBack），全部展示后显示"收取"按钮，点击将所有奖励批量写入背包，部分代码如下：Action<string> onEventTriggered = (eventName) => { if (eventName == "SpawnAwards" && !hasAwardsStarted) { hasAwardsStarted = true; GenerateItemAwardsAsync(token).Forget(); } }; // GenerateItemAwardsAsync内部：逐件生成，每件间隔0.15秒 go.transform.DOScale(1f, 0.25f).SetEase(Ease.OutBack); await

高度疑似
AIGC
(99.78%)

高度疑似
AIGC
(99.72%)

高度疑似
AIGC (100%)

高度疑似
AIGC
(99.99%)

高度疑似
AIGC
(86.69%)

轻度疑似
AIGC
(65.13%)

UniTask.Delay(TimeSpan.FromSeconds(0.15f), cancellationToken: token);

为防止动画事件异常导致奖励不展示，系统设计了双重保底：若等待2秒仍未收到SpawnAwards事件则直接执行奖励生成；若Animator组件不存在则延迟0.5秒后直接执行。失败分支：自动计时
 CombatRuleTable.DefeatAutoReturnDelaySeconds秒后淡出界面并切换回探索流程，无需玩家操作。所有异步操作均由CancellationTokenSource管理，OnClose时立即取消所有挂起任务。5.8.6 世界地图UI实现 地图界面如图5.17所示：图 5.17 地图界面 OverworldUI打开时遍历varMapItemUI容器内所有MapItemUI对象，读取其挂载的MapItemSceneldHolder组件获取场景ID并初始化地图标记。打开地图时通过
 PlayerInputManager.SetCursorLock(false)解锁鼠标，关闭时重新锁定，保证探索状态下鼠标行为的一致性。5.8.7 星盘UI实现 StarPhoneUI继承自StateAwareUIForm，同时订阅局外（OutOfGame）、局内（InGame）和探索（Exploration）三种状态的进入/离开事件，在多种游戏阶段均保持显示，实现跨状态常驻HUD效果。探索状态的离开事件故意不触发隐藏，使星盘在整个局内流程中始终可见。

5.8.8 UI系统实现总结 本节实现的各UI模块特征汇总如表5.1所示：UI模块 异步方案 动画方案 事件方案 对象池 主菜单UI — DOTween Sequence 按钮直连 — 战前准备UI UniTask+CTS DOTween错落弹出 C#事件 棋子卡对象池 战斗UI UniTask.Yield DOTween进场动画 GF.Event 卡牌对象池 战斗结算UI UniTask+CTS DOScale OutBack AnimationEvent — 世界地图UI — — 按钮直连 — 星盘UI — — GF.Event — 表5.1 UI模块实现特征对比 所有异步方法均通过GetCancellationTokensOnDestroy或手动CancellationTokenSource绑定UI生命周期，保证界面关闭后不产生空引用异常。DOTween动画在OnClose时统一Kill，防止对象池复用后残留Tween。5.9 本章小结

本章详细介绍了Clash of Gods游戏系统各核心模块的实现过程。技术美术系统实现了URP渲染管线配置、三套自定义着色器（描边、溶解、卡通渲染）、粒子特效对象池管理和基于Animation Event的动画同步机制。战斗系统实现了基于UniTask的异步战斗流程管理、棋子实体管理、完整的Buff生命周期管理、技能工厂模式、AI决策缓存机制、非线性伤害计算公式和脱战系统。物品系统实现了物品工厂、背包增删改查操作、装备系统、拖拽交互状态机和物品效果分发执行。卡牌系统实现了手牌管理和基于工厂字典的效果执行引擎。探索系统实现了五状态AI状态机、双层视野检测、战斗触发检测和精英敌人广播机制。数据配置系统实现了并行异步加载、静态缓存机制。性能优化方面，通过对象池、检测频率控制、NonAlloc物理检测、GPU Instancing、LOD系统和纹理压缩等手段，确保游戏在PC端的流畅运行。

各系统的实现均严格遵循第4章的设计方案，通过事件驱动实现模块解耦，通过配置表驱动实现数据与代码分离，通过UniTask实现高效的异步编程，保证了系统的可维护性、可扩展性和运行性能。6 系统测试 本章介绍Clash of Gods游戏系统的测试工作，包括测试环境配置、功能测试、性能测试、兼容性测试以及测试结果分析。6.1 测试环境 6.1.1 硬件环境 设备类型 处理器 显卡 内存 存储 操作系统 开发PC Intel Core i7-12700K NVIDIA RTX 3060 32GB DDR4 NVMe SSD Windows 11 测试PC（高配） Intel Core i7-13700K NVIDIA RTX 4070 32GB DDR5 NVMe SSD Windows 11 测试PC（中配） Intel Core i5-12400F NVIDIA GTX 1660 Super 16GB DDR4 SATA SSD Windows 10 测试PC（低配） Intel Core i5-10400 NVIDIA GTX 1050 Ti 8GB DDR4 SATA HDD Windows 10

表6.1 测试硬件环境 6.1.2 软件环境 软件 版本 Unity Editor 2022.3.10f1 LTS GameFramework 2023.09 UniTask 2.3.3 DOTween 1.2.745 .NET Framework 4.7.1 NVIDIA驱动 546.33+ 表6.2 测试软件环境 6.2 功能测试 6.2.1 测试方法与策略 功能测试采用黑盒测试方法，按照功能模块划分测试用例，验证各功能是否符合需求规格。测试策略如下：等价类划分：对输入数据进行等价类划分，选取典型值进行测试。边界值分析：重点测试边界条件，如背包满时添加物品、Buff层数达到上限等。场景测试：模拟真实游戏场景，测试多个功能的协同工作。6.2.2 战斗系统功能测试 测试编号 测试项目 测试输入 预期结果 测试结果 TC-C01 战斗开始 触发遭遇战 进入战斗场景，玩家获得先手Buff 通过 TC-C02 棋子攻击 棋子进入攻击范围 棋子自动攻击目标，播放攻击动画 通过

TC-C03 暴击判定 暴击率100%的棋子攻击 每次攻击均触发暴击，伤害为暴击倍率 通过 TC-C04 棋子死亡 棋子HP降至0 播放死亡动画，从战场移除 通过 TC-C05 战斗胜利 所有敌方棋子死亡 显示胜利界面，发放奖励 通过 TC-C06 战斗失败 所有己方棋子死亡 显示失败界面，扣除惩罚 通过 表6.3 战斗系统测试用例 6.2.3 Buff系统功能测试 测试编号 测试项目 测试输入 预期结果 测试结果 TC-B01 Buff添加 对棋子施加攻击力提升Buff 棋子攻击力增加，Buff图标显示 通过 TC-B02 Buff叠加 对已有Buff的棋子再次施加同类Buff Buff层数增加，效果叠加 通过 TC-B03 Buff到期 等待Buff持续时间结束 Buff自动移除，属性恢复 通过 TC-B04 Buff上限 叠加超过最大层数的Buff 层数保持在最大值，不再增加 通过

TC-B05 持续伤害Buff 施加灼烧Buff 每回合扣除固定HP，持续指定回合 通过 表6.4 Buff系统测试用例 6.2.4 技能系统功能测试 测试编号 测试项目 测试输入 预期结果 测试结果 TC-S01 普通攻击 棋子执行普通攻击 对目标造成正确伤害，播放攻击动画 通过 TC-S02 技能冷却 技能使用后立即再次使用 技能处于冷却中，无法使用 通过 TC-S03 AOE技能 使用范围技能 范围内所有敌人受到伤害 通过 TC-S04 条件Buff技能 后羿有烈焰箭Buff时攻击 攻击附加灼烧效果 通过 TC-S05 投射物技能 使用投射物攻击 投射物飞向目标，命中后造成伤害 通过 表6.5 技能系统测试用例 6.2.5 AI系统功能测试 测试编号 测试项目 测试输入 预期结果 测试结果 TC-A01 目标选择 多个敌人在不同距离 AI选择距离最近的敌人攻击 通过

TC-A02 移动追击 目标超出攻击范围 AI向目标移动直到进入攻击范围 通过 TC-A03 目标死亡 当前目标死亡 AI重新选择新目标 通过 TC-A04 无目标 所有敌人死亡 AI停止行动，进入待机状态 通过 表6.6 AI系统测试用例 6.2.6 物品系统功能测试 测试编号 测试项目 测试输入 预期结果 测试结果 TC-I01 添加物品 获得可堆叠物品 物品添加到背包，数量正确 通过 TC-I02 物品堆叠 获得已有的可堆叠物品 数量叠加，不新建格子 通过 TC-I03 背包已满 背包已满时获得物品 提示背包已满，物品未添加 通过 TC-I04 使用消耗品 使用恢复HP的消耗品 HP恢复，物品数量减少 通过 TC-I05 装备物品 将装备拖拽到装备槽 装备成功，属性生效 通过

表6.7 物品系统测试用例 6.2.7 背包系统功能测试 测试编号 测试项目 测试输入 预期结果 测试结果 TC-P01 打开背包 点击背包按钮 背包界面打开，显示当前物品 通过 TC-P02 物品详情 点击物品格子 显示物品详细信息 通过 TC-P03 整理背包 点击整理按钮 物品按类型和品质排序 通过 TC-P04 分页显示 背包物品超过一页 正确分页显示，翻页功能正常 通过 表6.8 背包系统测试用例 6.2.8 拖拽交互功能测试 测试编号 测试项目 测试输入 预期结果 测试结果 TC-D01 物品移动 拖拽物品到空格子 物品移动到目标格子 通过 TC-D02 物品交换 拖拽物品到有物品的格子 两个物品位置互换 通过 TC-D03 装备物

高度疑似
AIGC
(99.85%)

高度疑似
AIGC
(88.06%)

轻度疑似
AIGC
(53.11%)

高度疑似
AIGC
(96.26%)

高度疑似
AIGC
(85.18%)

高度疑似
AIGC
(90.46%)

品 拖拽装备到装备槽 装备成功，属性更新 通过

TC-D04 丢弃物品 拖拽物品到丢弃区域 弹出确认对话框，确认后物品移除 通过 表6.9 拖拽交互测试用例 6.2.9 卡牌系统功能测试 测试编号 测试项目 测试输入 预期结果 测试结果 TC-K01 获得卡牌 净化Boss敌人 获得对应卡牌，添加到手牌 通过 TC-K02 使用卡牌 战斗中使用战吼卡 所有友方棋子攻击力提升 通过 TC-K03 卡牌效果 使用削弱卡 所有敌方棋子攻击力降低 通过 表6.10 卡牌系统测试用例 6.2.10 探索系统功能测试 测试编号 测试项目 测试输入 预期结果 测试结果 TC-E01 敌人巡逻 观察敌人行为 敌人在巡逻范围内随机移动 通过 TC-E02 敌人警觉 进入敌人视野 敌人进入警戒状态，显示警戒UI 通过 TC-E03 敌人追击 警觉度满后停留 敌人开始追击玩家 通过

TC-E04 偷袭触发 从敌人背后接近 显示偷袭提示，可选择Debuff 通过 TC-E05 脱战成功 战斗中尝试脱战 按概率成功脱战，返回探索 通过 TC-E06 净化系统 击败Boss后净化 显示净化UI，获得卡牌 通过 表6.11 探索系统测试用例 6.2.11 UI系统功能测试 测试编号 测试项目 测试输入 预期结果 测试结果 TC-U01 UI打开动画 打开背包界面 界面以动画方式弹出 通过 TC-U02 UI关闭动画 关闭背包界面 界面以动画方式收起 通过 TC-U03 Buff图标更新 棋子获得Buff Buff图标实时显示在棋子头顶 通过 TC-U04 伤害飘字 棋子受到伤害 伤害数字从棋子位置飘出 通过 表6.12 UI系统测试用例 6.3 性能测试 6.3.1 帧率测试 在不同场景下测试游戏帧率，结果如表6.13所示： 测试场景 高配PC (RTX 4070) 中配PC (GTX 1660S) 低配PC (GTX 1050 Ti) 主菜单 60 FPS 60 FPS 60 FPS 探索场景 (无战斗) 60 FPS 60 FPS 55 FPS 战斗场景 (4v4) 60 FPS 60 FPS 45 FPS 战斗场景 (8v8) 60 FPS 55 FPS 32 FPS 表6.13 帧率测试结果 测试结果表明，在高配和中配PC上游戏运行流畅，帧率稳定在60 FPS附近。低配PC在4v4战斗场景下帧率为45 FPS，满足基本可玩性要求；在8v8复杂战斗场景下帧率降至32 FPS，后续可通过降低粒子特效质量和LOD优化进一步提升。

6.3.2 内存占用测试 测试场景 内存占用 游戏启动后 180 MB 进入探索场景 280 MB 进入战斗场景 350 MB 战斗结束返回探索 290 MB 长时间游戏 (1小时) 310 MB 表6.14 内存占用测试结果 内存占用在各场景下均在500MB以内，满足性能需求。长时间游戏后内存无明显增长，说明内存管理良好，无明显内存泄漏。 6.3.3 加载时间测试 加载项目 高配PC 中配PC 低配PC 游戏启动到主菜单 1.5s 2.3s 3.8s 进入探索场景 1.2s 1.8s 3.1s 进入战斗场景 0.8s 1.3s 2.4s 表6.15 加载时间测试结果 所有加载时间均在5秒以内，满足性能需求。高配和中配PC加载速度较快，低配PC因使用机械硬盘 (HDD)，加载时间相对较长，但仍在可接受范围内。 6.3.4 压力测试 在战斗场景中同时放置16个棋子 (8v8) 进行压力测试，持续战斗30分钟，观察游戏稳定性。测试结果：游戏运行稳定，无崩溃或异常退出，帧率保持在可接受范围内，内存无异常增长。

6.4 兼容性测试 6.4.1 操作系统兼容性测试 操作系统 测试设备 测试结果 备注 Windows 11 高配PC 通过 功能正常，帧率稳定 Windows 10 中配PC 通过 功能正常 Windows 10 低配PC 通过 复杂场景帧率较低 表6.16 操作系统兼容性测试结果 6.4.2 分辨率兼容性测试 分辨率 比例 测试结果 1920×1080 16:9 通过，UI布局正常 2560×1440 16:9 通过，UI自适应正常 3840×2160 16:9 通过，UI自适应正常 1280×720 16:9 通过，UI布局正常 2560×1080 21:9 通过，UI自适应正常 表6.17 分辨率兼容性测试结果 测试覆盖了从720p到4K的主流PC分辨率，包括21:9超宽屏，UI均能正确自适应。 6.4.3 输入方式兼容性测试 键盘鼠标操作：在PC端测试键盘和鼠标的交互操作，包括WASD移动、鼠标点击、物品拖拽、右键菜单、滚轮缩放等，所有操作响应正常，交互体验流畅。所有按键输入通过PlayerInputManager统一管理，支持自定义按键映射。

6.5 测试结果分析 6.5.1 功能测试结果总结 功能测试共设计并执行测试用例52个，其中战斗系统6个、Buff系统5个、技能系统5个、AI系统4个、物品系统5个、背包系统4个、拖拽交互4个、卡牌系统3个、探索系统6个、UI系统4个，另含边界条件测试 (背包满、Buff上限等) 6个，全部通过，通过率100%。 6.5.2 性能测试结果总结 性能测试结果表明，游戏在高配和中配PC上运行流畅，帧率稳定在55-60 FPS。低配PC在常规场景下帧率满足可玩性要求，在8v8复杂战斗场景下帧率有所下降，后续可通过降低特效质量进一步优化。内存占用控制在合理范围内，无内存泄漏问题。加载时间满足需求规格要求。 6.5.3 发现的问题与修复 测试过程中发现并修复了以下问题：

问题1：战斗结束后Buff图标未清除。原因：战斗结束时未触发BuffRemovedEventArgs事件。修复：在CombatManager.EndCombat()中主动清除所有Buff并触发事件。 问题2：背包满时拖拽物品到其他格子导致数据错误。原因：拖拽逻辑未检查目标格子是否有效。修复：在拖拽结束时增加有效性检查。 问题3：低配PC战斗场景帧率较低。原因：粒子特效数量过多。修复：增加特效质量分级设置，低配环境自动降低粒子数量和特效复杂度。 6.5.4 测试结论 通过全面的功能测试、性能测试和兼容性测试，Clash of Gods游戏系统的各项功能均符合需求规格，性能表现满足目标要求，在主流PC配置上具有良好的兼容性。系统整体质量达到预期目标。

6.6 本章小结 本章对Clash of Gods游戏系统进行了全面的测试工作。在功能测试方面，针对战斗系统、Buff系统、技能系统、AI系统、物品系统、背包系统、拖拽交互、卡牌系统、探索系统和UI系统共设计了52个测试用例，全部通过。在性能测试方面，帧率、内存占用和加载时间均满足需求规格。在兼容性测试方面，游戏在Windows 10/11平台和多种PC分辨率 (720p至4K及21:9超宽屏) 下均能正常运行。测试过程中发现的问题均已修复，系统整体质量达到预期目标。 7 总结与展望 7.1 工作总结 7.1.1 完成的主要工作 本文围绕多玩法融合游戏Clash of Gods的设计与实现，完成了以下主要工作：第一，进行了系统的需求分析。通过分析多玩法融合游戏的市场现状和技术发展趋势，明确了游戏的功能需求和非功能需求，建立了用例模型，为系统设计提供了依据。

第二，设计了完整的系统架构。采用分层架构设计，将系统划分为基础框架层、核心系统层、业务逻辑层和表现层。通过事件驱动架构实现模块解耦，通过配置表驱动设计实现数据与代码分离。第三，实现了核心游戏系统。包括：基于自走棋机制的战斗系统 (棋子AI、技能系统、Buff系统、伤害计算)；支持多种物品类型的物品背包系统 (堆叠机制、拖拽交互、词条系统)；基于命令模式的策略卡系统；具有AI状态机的探索系统 (视野检测、战斗触发、净化系统)。第四，实现了技术美术系统。包括URP着色器 (描边、溶解、卡通渲染)、粒子特效系统和动画系统，为游戏提供了良好的视觉表现。

第五，进行了全面的系统测试。通过功能测试、性能测试和兼容性测试，验证了系统的正确性和稳定性，并修复了测试中发现的问题。 7.1.2 系统实现成果 本文实现的Clash of Gods游戏系统具有以下成果：功能完整性：实现了探索、战斗、物品管理、卡牌策略等核心玩法，游戏流程完整，各系统协同工作正常。技术先进性：采用了UniTask异步编程、GameFramework框架等先进技术，系统架构现代化。性能表现：在PC端保持60 FPS的流畅帧率，内存占用控制在

高度疑似
AIGC
(99.33%)

中度疑似
AIGC (73.1%)

中度疑似
AIGC (73.1%)

高度疑似
AIGC
(99.93%)

高度疑似
AIGC
(96.68%)

高度疑似
AIGC
(99.96%)

350MB以内，加载时间满足需求。可扩展性：模块化设计使得新增棋子、物品、卡牌等内容只需添加配置和少量代码，扩展成本低。7.2 主要贡献 7.2.1 理论贡献 本文在理论层面的主要贡献包括：

提出了将RPG、肉鸽、搜打撤、自走棋、卡牌五种玩法有机融合的设计方案，为同类游戏的系统设计提供了新思路[6][7][17][18]。Apken等（2012）研究了多类型游戏机制的参数化设计[17]，本文的设计方案在此基础上进行了实践验证。总结了基于GameFramework框架的游戏系统架构设计方法[5][24]，包括分层架构、事件驱动、模块化设计等最佳实践。探索了配置表驱动开发在游戏开发中的应用模式[11][12]，为游戏开发中的数据管理提供了参考。7.2.2 实践贡献 本文在实践层面的主要贡献包括：实现了一套完整的多玩法融合游戏系统，验证了技术方案的可行性。积累了Unity PC游戏开发的工程实践经验，包括性能优化、异步编程等方面。产出了可复用的代码资产，各功能模块具有较高的通用性，可在其他项目中参考使用。

7.3 存在的问题 7.3.1 系统局限性 当前系统存在以下局限性：视线遮挡检测缺失：探索系统的视野检测仅基于距离和角度，未考虑障碍物遮挡，可能导致敌人“透视”发现玩家的情况。单机游戏限制：当前系统为单机游戏，缺乏多人联机功能，限制了游戏的社交属性和长期留存。内容量不足：游戏目前的棋子、物品、卡牌数量有限，长期游戏体验有待丰富。低端设备优化不足：在低端设备上，复杂战斗场景的帧率仍有提升空间。7.3.2 待改进之处 基于上述局限性，以下方面有待改进：视线遮挡：添加射线检测（Raycast）验证视线，实现更真实的视野检测。AI记忆系统：敌人丢失玩家后保留一段时间的位置信息，增加追击的真实感。

平衡性调整：通过更多的测试和数据分析，优化游戏数值平衡。7.4 未来展望 7.4.1 功能扩展方向 在功能扩展方面，未来可以考虑以下方向：多人联机：引入实时对战或异步对战功能，增加游戏的社交属性和竞技性。内容扩展：增加更多棋子、物品、卡牌和地图，丰富游戏内容，延长游戏生命周期。剧情系统：添加完整的剧情系统，通过对话、过场动画等方式讲述游戏世界观和故事。成就系统：引入成就和排行榜系统，为玩家提供更多长期目标。7.4.2 技术优化方向 在技术优化方面，未来可以考虑以下方向：AI增强：引入更复杂的AI决策算法，如行为树或机器学习，使棋子AI更加智能。渲染优化：进一步优化渲染管线，支持更高质量的光照和阴影效果，同时保持移动端性能。自动化测试：引入自动化测试框架，提高测试效率和覆盖率，保证版本质量。

参考文献 章善举,刘红卫,王婷.基于双缓冲区技术的图像渲染和显示优化方案的研究[J].中国新通信,2025,27(17):20-22. Hussain A, Shakeel H, Hussain F. Unity game development engine: A technical survey[J]. University of Sindh Journal of Information, 2020, 2(2): 35-42. Menard M, Wagstaff B. Game development with Unity[M]. 2nd ed. Searcher.com.br, 2012: 1-118. Maharjan B. Puzzle game using Android MVVM Architecture[D]. Lappeenranta: Centria University of Applied Sciences, 2018. Sripada SK, Reddy YR, Khandelwal S. Architecting an extensible framework for gamifying software engineering concepts[C]// IEEE. 2016 IEEE Frontiers in Education Conference. Piscataway: IEEE, 2016: 1-9. Sepänmaa T. Procedural level generation in 2D roguelite games[D]. Oulu: Oulu University of Applied Sciences, 2024. Gellel A, Sweetser P. A hybrid approach to procedural generation of roguelike video game levels[C]// ACM. Proceedings of the 2020 ACM Conference on the Foundations of Digital Games. New York: ACM, 2020: 1-10. Xu Y, Chen X, Zhang H, Wang L, et al. Lineup Mining and Balance Analysis of Auto Battler[C]// ACM. Proceedings of the 19th ACM International Conference on Multimedia. New York: ACM, 2020: 1-9. García-Sánchez P, Tonda A, Mora AM. Automated Playtesting in Collectible Card Games Using Evolutionary Algorithms: A Case Study in Hearthstone[J]. Knowledge-Based Systems, 2018, 153: 133-146. Jagdale D, Koli N, Mali N, Prajapati P. Finite State Machine in Game Development[J]. International Journal of Advanced Research in Science, Communication and Technology, 2021, 8(1): 425-429. Barros GAB, Green MC, Liapis A, Togelius J. Data-driven design: A case for maximalist game design[C]// ACM. Proceedings of the 13th International Conference on the Foundations of Digital Games. New York: ACM, 2018: 1-10. Pfau J, Seif El-Nasr M. On video game balancing: Joining player- and data-driven analytics[J]. ACM Games: Research and Practice, 2024, 1(1): 1-25. Aversa D, Dickinson C. Unity Game Optimization: Enhance and extend the performance of all aspects of your Unity games[M]. 3rd ed. Packt Publishing, 2019: 1-404. Dickinson C. Unity 2017 Game Optimization: Optimize All Aspects of Unity Performance[M]. 2nd ed. Packt Publishing, 2017: 1-376. Llanos SC, Jørgensen K. Do players prefer integrated user interfaces? A qualitative study of game UI design issues[C]// DiGRA. Proceedings of DiGRA 2011 Conference: Think Design Play. DiGRA, 2011: 1-16. Aversa D, Dickinson C. Unity Game Optimization[M/OL]. 3rd ed. <https://books.google.com.hk/books?id=f1jBDwAAQBAJ>, [2024-05-19]. Apken D, Landwehr H, Herrlich M, Krause M, et al. Parameterizable game mechanics of different genres for evaluation[C]// Springer. International Conference on Entertainment Computing. Berlin: Springer, 2012: 127-139. 余汪宇.电子游戏中的随机性设计研究[D].南京:东南大学,2021. 杨梓.《刀塔自走棋》为什么好玩?[J].电子竞技,2018(23):42-45. Unity Technologies. Unity User Manual[M/OL]. <https://docs.unity3d.com/Manual/index.html>, [2024-05-19]. Unity Technologies. Universal Render Pipeline Documentation[M/OL]. <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.0/manual/index.html>, [2024-05-19]. Cysharp. UniTask - Providing an efficient allocation free async/await integration for Unity[CP/OL]. <https://github.com/Cysharp/UniTask>, [2024-05-19]. Demigiant. DOTween Documentation[CP/OL]. <http://dotween.demigiant.com/documentation.php>, [2024-05-19]. Ellanjiang. Game Framework for Unity[CP/OL]. <https://github.com/Ellanjiang/UnityGameFramework>, [2024-05-19]. Unity Technologies. NavMesh Building Components[M/OL]. <https://docs.unity3d.com/Manual/NavMesh-BuildingComponents.html>, [2024-05-19]. Unity Technologies. GPU Instancing[M/OL]. <https://docs.unity3d.com/Manual/GPUInstancing.html>, [2024-05-19]. Unity Technologies. Level Of Detail (LOD)[M/OL]. <https://docs.unity3d.com/Manual/LevelOfDetail.html>, [2024-05-19]. Nystrom R. Game Programming Patterns[M/OL]. <https://gameprogrammingpatterns.com/contents.html>, [2024-05-19]. OSCHINA. 2024年游戏技术现状调查报告

高度疑似
AIGC
(93.23%)

高度疑似
AIGC
(99.95%)

高度疑似
AIGC
(98.59%)